



MORPHOIA

MORPHOIA ENGINE

CAHIER DES CHARGES TECHNIQUE

Spécification normative pour lancer la réalisation de la plateforme ouverte d'interopérabilité CAO, données 3D, imagerie médicale, simulation, IA et HPC.

VERSION DU DOCUMENT 0.1 | 6 AOÛT 2026

Auteur et porteur du projet : Dr Olivier Ami

Référence d'architecture : Morphoia Engine v0.2

Statut : prêt pour lancement P0 sous réserve d'approbation du présent cahier des charges

Décision contractuelle en une page

Le présent cahier des charges transforme l'architecture de référence Morphoia Engine v0.2 en un contrat de réalisation mesurable. Il autorise le lancement immédiat de P0, tout en empêchant le projet de dériver vers une nouvelle suite CAO, un moteur de rendu généraliste ou un framework IA.

Arbitrage directeur

Construire une couche ouverte de continuité et de preuve entre B-Rep, meshes, points, DICOM, volumes, champs, tenseurs, MVX et HPC. Réemployer les meilleurs moteurs existants derrière des contrats stables. Mesurer toute perte. Conserver un chemin CPU intégralement ouvert.

Décision	Spécification retenue	Conséquence de réalisation
Tranches	P0 et P1 fermes ; P2 à P4 optionnelles	Le financement suit des gates Go/No-Go plutôt qu'une promesse globale.
Socle	C++20, ABI C11, Python 3.13 de référence	Compatibilité 3.12 ; Python 3.14 après validation de la pile native.
Données	IR JSON canonique + CAS SHA-256	Les gros payloads restent dans leurs formats spécialisés et sont adressés par contenu.
CAO	OCCT/XDE + profil STEP/AP242	Fidélité testée ; aucune promesse AP242 universelle.
Scientifique	VTK/ITK/PDAL ; MEDCoupling optionnel	Réemploi direct avec paquets séparés lorsque la licence l'impose.
CAE	SALOME 9.16 via agent isolé dès P0	Aucune dépendance du cœur à KERNEL, CORBA, Qt ou Python SALOME.
GPU	Workers CPU, CUDA et HIP ; SYCL Tier 2	Même source portable, binaires ciblés par plateforme.
IA	DLPack + Arrow ; PyTorch/ONNX externes	Aucun framework d'entraînement Morphoia et copies instrumentées.
HPC	Bundles Slurm, Spack, Apptainer, ADIOS2	Exécution rejouable sans service central obligatoire.
MVX	Axe expérimental indépendant du socle	Un échec bloque le JEPA, pas l'interopérabilité Morphoia.
Licence	Apache-2.0 OR MIT	Projet gouverné sans objectif commercial, mais licence réellement open source.

Autorisation de lancement

P0 peut commencer dès approbation de ce document. Aucune phase ultérieure ne peut être engagée sur simple avancement calendaire : chaque tranche exige ses preuves, ses artefacts hashés et l'acceptation formelle de son gate.

Contrôle du document

Champ	Valeur
Titre	Morphoia Engine - Cahier des charges technique
Version du document	0.1
Date de référence	6 août 2026
Auteur et porteur	Dr Olivier Ami
Référence d'architecture	Morphoia Engine - Architecture de référence v0.2
Empreinte de la référence	8519bba50ab9a05d469780ec074588034714d83d24ef09404705d292bfa02368
Statut	Baseline proposée pour lancement P0
Public	Développeurs C++/Python, experts CAO/CAE, DICOM, GPU/HPC, QA, sécurité et partenaires
Licence du document	CC BY 4.0, sous réserve de validation finale du titulaire
Code original attendu	Apache-2.0 OR MIT
Horizon	P0 : 6 semaines ; vertical slice : mois 4 ; v1 crédible : 15 à 18 mois

Statut et ordre de préséance

- Ce document est normatif pour le périmètre, les exigences, les preuves, les gates et les livrables.
- L'architecture v0.2 demeure la justification des arbitrages. En cas de conflit, ce cahier des charges prévaut pour la réalisation.
- Les ADR acceptés précisent les décisions sans pouvoir diminuer une exigence MUST sans waiver approuvé.
- Les schémas versionnés, profils de formats et fixtures deviennent les sources exécutables de vérité au fur et à mesure de P0.
- Les faits de versions et licences doivent être reverrouillés dans les lockfiles et la SBOM au démarrage de P0.

Langage normatif

Terme	Portée	Traitement d'un écart
DOIT / MUST	Obligation contractuelle	Non-conformité ; waiver écrit, daté et limité requis.
DEVRAIT / SHOULD	Recommandation forte	Écart argumenté, mesuré et approuvé dans un ADR.
PEUT / MAY	Option	Aucun engagement avant décision de phase.
Hors périmètre	Attente explicitement refusée	Toute demande devient un change request chiffré.

Catalogue d'exigences

Le catalogue contient **320 exigences atomiques** réparties en 20 familles. Chaque exigence possède un identifiant immuable, une priorité, une phase et une méthode de preuve. Les preuves sont T = test, I = inspection, A = analyse, D = démonstration.

Indicateur	Valeur
Exigences MUST	305
Exigences SHOULD	10
Exigences MAY	5
Familles normatives	20
Source de vérité attendue	spec/requirements/requirements.yaml
Chaîne de traçabilité	REQ-ID -> ADR/composant -> issue/PR -> TEST-ID -> résultat CI -> artefact -> PV

Règle de recette

Une démonstration visuelle n'est jamais une preuve suffisante. Toute exigence MUST doit être reliée à une preuve vérifiable, conservée et hashée.

Table des matières

Décision contractuelle en une page	2
Contrôle du document	3
Statut et ordre de préséance	3
Langage normatif	3
Catalogue d'exigences	3
Table des matières	4
1. Objet de la consultation	7
1.1 Objectifs de produit	7
1.2 Hors périmètre impératif	7
1.3 Utilisateurs et rôles	7
1.4 Cas d'usage prioritaires	8
1.5 Indicateurs de succès v1	8
2. Modèle contractuel et tranches	8
3. Architecture cible	9
3.1 Plan de contrôle et data plane	9
3.2 Topologie des dépôts	9
3.3 Modules et responsabilités	10
3.4 Pile logicielle par couche	10
3.5 Profils de déploiement	10
4. Architecture SALOME 9.16	10
4.1 Cycle du protocole	11
4.2 Règles de frontière	11
5. Exigences transversales	12
5.1 Périmètre et principes	12
5.2 Architecture et frontières	12
5.3 Développement et toolchain	13
5.4 API, ABI et versionnement	13
6. Données, IR, CAS et formats	14
6.1 Morphoia IR	14
6.2 Stockage adressé par contenu	14
6.3 Profils d'entrée/sortie	15
6.4 DICOM et confidentialité	15
7. Visualisation, calcul, IA et MVX	16
7.1 Visualisation et streaming	16
7.2 Simulation et portabilité GPU	16
7.3 Interopérabilité IA	17
7.4 Expérimentation MVX	17
8. Extensions, SALOME et HPC	18
8.1 Plug-ins et connecteurs	18

8.2 Profil SALOME 9.16	18
8.3 Exécution HPC	19
9. Sécurité, performance, qualité et gouvernance	19
9.1 Sécurité et supply chain	19
9.2 Déterminisme et performance	20
9.3 QA, CI et recette	20
9.4 Licences	21
9.5 Gouvernance	21
10. Morphoia IR v0.1	23
10.1 Identité et canonicalisation	23
10.2 Événements de fidélité	23
10.3 Bloc SALOME	23
11. ABI C et SDK	23
11.1 Taxonomie des diagnostics	24
12. CLI et API Python	24
13. Job bundle et profil de site	24
13.1 États d'exécution	24
14. Profils de formats initiaux	25
15. Lots de travaux	26
15.1 Dépendances entre lots	26
16. Plan P0 - six semaines	26
16.1 Backlog initial - dix jours SALOME	26
16.2 Definition of Done P0	27
17. Gates Go/No-Go	27
17.1 Gate G1 - fin P0	27
17.2 Gates G2 à G5	27
18. Livrables contractuels	27
19. Équipe, RACI et charge	28
19.1 Répartition des 60 personnes-mois	28
20. Budget indicatif et leviers de réduction	28
20.1 Économies déjà intégrées	28
21. Stratégie de vérification	29
21.1 Chaîne de traçabilité	29
21.2 Source de vérité des exigences	29
22. Corpus de conformité	29
22.1 Manifeste d'un actif de conformité	29
22.2 Mesures obligatoires par domaine	30
23. Classification et acceptation des anomalies	30

23.1 Waivers 30

24. Matrice CI et release 30

24.1 Release checklist 30

25. Registre de risques 31

26. ADR initiaux 31

27. Dépendances externes à sécuriser 32

27.1 Conditions préalables au jour 1 32

28. Plan d'action des 30 premiers jours 32

Annexe A. Pile logicielle de référence 33

Annexe B. Matrice de formats et objectifs 34

Annexe C. Résumé du catalogue d'exigences 34

Annexe D. Glossaire 35

Annexe E. Références officielles 36

01

Mission et périmètre

Besoin, utilisateurs, cas prioritaires et définition du succès

1. Objet de la consultation

L'objet est de spécifier, concevoir, implémenter, tester, documenter et transférer Morphoia Engine, une infrastructure headless qui maintient une continuité vérifiable entre géométrie exacte, données scientifiques, imagerie médicale, simulation, intelligence artificielle et calcul haute performance.

Le titulaire ou l'équipe de réalisation doit livrer du code open source, des contrats stables, des profils de formats, des corpus de conformité, des preuves de performance et une exploitation reproductible. Le projet ne consiste pas à réécrire les logiciels verticaux existants.

1.1 Objectifs de produit

- Conserver simultanément plusieurs représentations d'un même objet et leur filiation.
- Rendre explicites unités, repères, tolérances, pertes, réparations, versions et autorité.
- Importer et dériver les familles prioritaires STEP/AP242, DICOM et LAS/LAZ, puis MED/VTXHDF.
- Exécuter les mêmes contrats sur CPU, NVIDIA, AMD et à terme SYCL, sans masquer les écarts numériques.
- Échanger les données avec les frameworks IA sans copie lorsque le runtime le permet et expliquer les copies résiduelles.
- Rejouer un job localement ou sur Slurm avec les mêmes entrées, hashes, paramètres et règles de validation.
- Utiliser SALOME comme backend CAE et oracle différentiel sans l'introduire dans le cœur.
- Valider séparément la faisabilité de l'encodage MVX sur un micro-corpus puis environ 1 000 objets.

1.2 Hors périmètre impératif

Refus	Motif
Nouveau noyau CAO généraliste	OCCT et les outils sources couvrent déjà la géométrie exacte.
Nouveau solveur multiphysique	PETSc, MFEM, FEniCSx, SOFA, OpenFOAM et autres restent spécialisés.
Nouveau framework de deep learning	PyTorch, ONNX Runtime et DLPack sont les frontières retenues.
Moteur de rendu généraliste maison	VTK/ParaView réduisent drastiquement le coût du MVP.
Import parfait de tous les formats propriétaires	Il dépend de SDK et licences externes non ouverts.
Compatibilité binaire universelle des plugins	Chaque hôte possède son ABI et son modèle de données.
Usage clinique revendiqué	La v1 reste recherche uniquement sans cycle réglementaire.
SaaS central obligatoire	Local, hors ligne et HPC doivent rester autonomes.
GUI desktop propre au MVP	CLI, notebooks et connecteurs hôtes sont prioritaires.
MeshGems ou SDK constructeur obligatoire	Le chemin de référence doit rester intégralement ouvert.

1.3 Utilisateurs et rôles

Rôle	Besoin principal	Interface prioritaire
Chercheur 3D/IA	Préparer, encoder, reconstruire, profiler et entraîner	Python, notebooks, CLI
Ingénieur CAO/CAE	Préserver géométrie, groupes, mesh, champs et conditions	FreeCAD/SALOME/ParaView + CLI
Chercheur médical	Conserver DICOM/FoR, dériver volumes et surfaces	Slicer, Python, VTK
Ingénieur HPC	Concrétiser, soumettre, checkpoint et diagnostiquer	CLI, bundles, Slurm, Spack
Développeur intégrateur	Étendre I/O, calcul et hôtes	ABI C, SDK, Python
Responsable qualité	Relier exigences, tests, résultats et waivers	Rapports, CI, corpus
Product Owner	Arbitrer périmètre, gates et acceptation	Tableau de conformité et PV

1.4 Cas d'usage prioritaires

ID	Scénario	Résultat observable	Gate
UC-01	STEP -> IR -> B-Rep/tessellation -> glTF/VTK	Unités, placements, assemblages, pertes et métriques vérifiables	P1
UC-02	DICOM -> IR -> volume/surface -> VTK	Graphe DICOM et Frame of Reference conservés	P1
UC-03	LAS/LAZ/COPC -> IR -> LOD	CRS, attributs, précision et streaming conservés	P1
UC-04	Objet -> LP3 MVX -> reconstruction	Métriques 3D et échecs catégorisés sur 30-50 puis ~1 000 objets	P0/P1
UC-05	Mesh/champ -> DLPack/Arrow -> IA -> annotation	Zéro copie démontré ou copie instrumentée	P1
UC-06	Même kernel sur CPU/CUDA/HIP	Résultats dans tolérance et profil bout-en-bout	P0
UC-07	Job bundle -> Slurm/Apptainer -> checkpoint	Rejeu sans service central et publication hashée	P1
UC-08	STEP -> SMESH -> MED -> IR	Agent SALOME isolé, groupes/champs conservés	P0/P1
UC-09	ParaView vs ParaVis	Structure des champs identique ou divergence qualifiée	P1

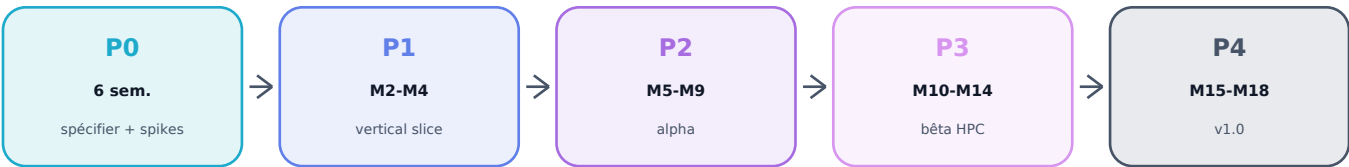
1.5 Indicateurs de succès v1

- Un tiers installe le socle depuis un tag public, sans compte privé ni composant propriétaire obligatoire.
- Un actif est importé, validé, dérivé et rejoué avec la même identité de contenu et un rapport de fidélité lisible.
- Le chemin CPU, les tests et la documentation fonctionnent hors ligne.
- Les workers CPU/CUDA/HIP partagent le contrat et publient leurs écarts numériques et coûts réels.
- Au moins trois cas utilisateurs externes sont entièrement reproductibles à la fin de P3.
- La suite de conformité peut être utilisée par une implémentation indépendante de Morphoia.

2. Modèle contractuel et tranches

Le programme est découpé en deux tranches fermes et trois tranches optionnelles. L'effort cumulé vient de l'architecture v0.2 et suppose une équipe moyenne de quatre à cinq ETP, le réemploi des briques retenues et l'absence de GUI propriétaire au MVP.

Tranche	Période	Effort cumulé	Objet	Décision
TF0 / P0	6 semaines	≈ 6 pers.-mois	Spécification exécutable et destruction des risques	Gate G1
TF1 / P1	Jusqu'au mois 4	≈ 16 pers.-mois	Vertical slice v0.1 démontrable	Gate G2
TO1 / P2	Mois 5-9	≈ 28 pers.-mois	Alpha multi-domaine et SDK v1-rc	Gate G3
TO2 / P3	Mois 10-14	≈ 40 pers.-mois	Bêta EuroHPC et trois cas externes	Gate G4
TO3 / P4	Mois 15-18	≈ 60 pers.-mois	ABI v1, profils publics et stabilisation	Gate G5



Affermissement Une tranche optionnelle n'est affermie que si le gate précédent est accepté, que les ressources matérielles sont disponibles et qu'aucune dépendance propriétaire obligatoire n'est apparue.

02

Architecture de réalisation

Frontières stables, modules, data plane et déploiements

3. Architecture cible

Interfaces	Python ABI C CLI notebooks
Orchestration	DAG transactions jobs provenance
Morphoia IR	manifestes unités repères pertes identités
Représentations	B-Rep mesh points voxels champs tenseurs MVX
Moteurs	OCCT VTK/ITK PDAL Kokkos/PETSc OpenUSD
Exécution	CPU CUDA HIP SYCL Slurm/EuroHPC

Morphoia conserve une sémantique commune sans imposer une représentation physique unique. Les bibliothèques spécialisées restent remplaçables ; seules les abstractions Morphoia et les profils de conformité sont exposés comme contrats durables.

3.1 Plan de contrôle et data plane

Plan	Contenu	Transport	Interdit
Contrôle local	Commandes, statuts, URI, hashes, limites, erreurs	C ABI, JSON, CLI	Meshes, voxels ou champs massifs
Contrôle distant	Même sémantique, authentification et reprise	gRPC optionnel P2	Nouvelle sémantique parallèle
Data plane fichier	STEP, DICOM, MED, XAO, VTKHDF, ADIOS2	CAS/POSIX/objet	Mutation in-place de l'autorité
Data plane mémoire	Tenseurs et colonnes	DLPack, Arrow C Data/Device	Copie cachée ou durée de vie implicite
Data plane IPC	Arrays et résultats inter-processus	Arrow IPC, shared memory, ADIOS2	JSON volumineux

3.2 Topologie des dépôts

Afin de réduire les temps de build, la duplication d'issues et le coût de versionnement, P0-P1 utilise un monorepo principal. Les séparations ne sont admises que pour une frontière juridique, de runtime ou de volume de données.

```
morphoia-engine/  
spec/{requirements,ir,protocols,format-profiles,numerical-profiles,adr}  
cpp/{core,capi,compute,io/{occt,vtk,itk,pdal,dicom}}  
python/ cli/ workers/{cpu,cuda,hip,sycl}  
hpc/ plugins/sdk/ tests/{unit,property,integration,e2e,fuzz,perf}  
packaging/ docs/ examples/  
morphoia-io-med/ # paquet LGPL dynamique séparé  
morphoia-salome-agent/ # runtime SALOME isolé  
morphoia-conformance/ # manifestes, générateurs, petits actifs
```

3.3 Modules et responsabilités

Module	Responsabilité	Dépendance obligatoire
spec	Exigences, IR, protocoles, profils, ADR, corpus	Aucune runtime
core	Identités, DAG, unités, provenance, CAS, diagnostics, transactions	C++20, STL interne
capi	ABI C, handles, allocateurs, vues, erreurs	core
python/cli	Orchestration, inspection, scripts et notebooks	capi
io-occt	STEP/AP242, B-Rep, assemblages, XDE	OCCT dynamique
io-vtk/itk/pdal/dicom	Meshes, volumes, points et objets médicaux	Une pile par module
io-med	MEDCoupling/MEDLoader, champs et remapping	Paquet LGPL séparé
compute/workers	Kokkos, kernels, profils CPU/CUDA/HIP/SYCL	Toolchain ciblée
hpc	Bundles, profils Slurm, Spack, Apptainer, checkpoints	Clients du site
salome-protocol	Schémas et client sans dépendance SALOME	core
salome-agent	GEOM/SHAPER/SMESH dans runtime 9.16	Distribution séparée
conformance	Fixtures, générateurs, invariants et comparaisons	Aucune autorité unique

3.4 Pile logicielle par couche

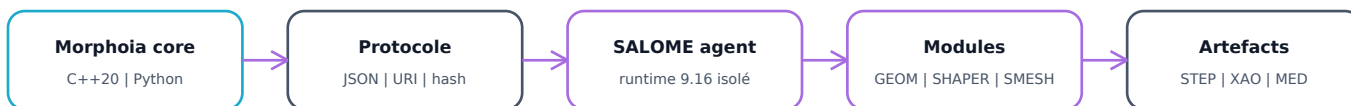
Couche	Choix de référence	Statut
Géométrie	OCCT/OCAF/XDE + profil STEP/AP242	Obligatoire P1
Scientifique	VTk/VTkHDF, ITK, PDAL	Obligatoire P1
Médical	DCMTK + GDCM/ITK ; DICOM autoritaire	Obligatoire P1
CAE	MEDCoupling direct optionnel ; SALOME 9.16 agent	P0-P1
Calcul	Kokkos/Kokkos Kernels ; PETSc/MFEM ensuite	Spike P0, extension P2
Rendu	VTk/ParaView ; ANARI/Vulkan/OpenUSD ensuite	MVP puis options
IA	DLPack, Arrow, PyTorch et ONNX Runtime externes	P0-P1
HPC	Slurm, MPI du centre, Spack, Apptainer, ADIOS2	P0-P3
Supply chain	REUSE/SPDX, SBOM, signatures, provenance	P0-P2

3.5 Profils de déploiement

Profil	Contenu	Garantie
Local OSS	Core, CPU, Python, VTK/ITK/PDAL, CAS local	Aucun compte, aucun runtime propriétaire
Workstation GPU	Worker CUDA ou HIP installé par recette	Backend optionnel et capacités détectées
SALOME local	Installation 9.16 utilisateur + agent	Aucune librairie SALOME dans core
HPC	Job bundle + Spack lock + Apptainer + profil de site	Stateless, checkpoint, pas de serveur central
Connecteurs	ParaView, Slicer, FreeCAD	Dépôt et matrice de versions séparés si nécessaire

4. Architecture SALOME 9.16

Réemploi direct : [morphoia-io-med](#) | [MEDCoupling/MEDLoader](#)



Oracle différentiel : divergence = analyse, jamais vérité automatique

SALOME est un backend CAE optionnel et un banc de comparaison dès P0. MEDCoupling peut être réutilisé directement dans un paquet LGPL séparé. SHAPER, GEOM et SMESH restent dans un agent hors processus. CORBA reste interne au runtime SALOME.

4.1 Cycle du protocole

```
ProbeCapabilities -> Submit -> Observe | Cancel -> Publish
```

```
request = {  
  protocol_version, request_id, idempotency_key, operation,  
  inputs: [{uri, sha256, size, media_type}],  
  parameters, limits, expected_outputs  
}
```

4.2 Règles de frontière

- Aucun TopoDS_Shape, objet CORBA, handle SALOME ou pointeur ne traverse le processus.
- STEP/AP242 est la frontière CAO générale ; XAO est spécialisé ; MED transporte meshes et champs.
- Les études HDF et dumps éventuels sont des artefacts de replay, pas l'identité canonique.
- Les données supérieures à 64 Kio devraient être externes ; les cas supérieurs à 2 Gio sont obligatoires en conformité.
- ParaVis est un oracle de structure et de rendu ; ParaView/VTK amont reste le chemin produit.
- MeshGems est absent du profil OSS. YACS et JOBMANAGER attendent un cas utilisateur P2.

03

Exigences normatives

Catalogue atomique, priorisé, phasé et vérifiable

5. Exigences transversales

Les tableaux suivants sont la baseline contractuelle. Les identifiants restent immuables. Toute évolution du texte conserve un historique de baseline et ne réutilise jamais un ID supprimé.

5.1 Périmètre et principes

Règles fondatrices et anti-dérive du projet.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-SCP-001	Morphoia DOIT être livré comme une infrastructure ouverte d'interopérabilité scientifique 3D, et non comme un moteur monolithique.	MUST	P0	I/T
MOR-SCP-002	L'architecture DOIT fédérer les représentations sans les aplatir dans un format binaire universel.	MUST	P0	I/T
MOR-SCP-003	Toute donnée DOIT déclarer explicitement son autorité, ses dérivés et les relations de provenance.	MUST	P0	I/T
MOR-SCP-004	Toute réparation, approximation, conversion avec perte ou changement de convention DOIT être explicite, mesuré et réversible lorsque possible.	MUST	P0	I/T
MOR-SCP-005	Le cœur DOIT fonctionner sans interface graphique et sans GPU d'affichage.	MUST	P0	I/T
MOR-SCP-006	Toute capacité essentielle DOIT disposer d'un chemin CPU open source, testable et documenté.	MUST	P0	I/T
MOR-SCP-007	Le fonctionnement local NE DOIT dépendre d'aucun service distant, compte commercial ou cloud payant.	MUST	P0	I/T
MOR-SCP-008	Les backends propriétaires DOIVENT rester facultatifs, installés séparément et absents de la distribution OSS de référence.	MUST	P0	I/T
MOR-SCP-009	Le produit v1 médical DOIT porter la mention recherche uniquement tant qu'aucun cycle réglementaire et aucune validation clinique ne sont conduits.	MUST	P0	I/T
MOR-SCP-010	L'ajout d'un nouveau domaine après P1 DOIT être conditionné par un cas utilisateur, un mainteneur et un corpus de conformité.	MUST	P2+	I
MOR-SCP-011	Les bibliothèques spécialisées DOIVENT rester remplaçables derrière des contrats Morphoia stables.	MUST	P0	I/T
MOR-SCP-012	Les gains de performance DOIVENT inclure I/O, transferts, synchronisations, compilation et orchestration.	MUST	P0	I/T
MOR-SCP-013	Le succès du socle Morphoia NE DOIT dépendre ni du succès scientifique de MVX ni d'un futur entraînement JEPA.	MUST	P0	I/T
MOR-SCP-014	Le positionnement public DEVRAIT utiliser la formulation Open interoperability fabric for 3D scientific computing.	SHOULD	P0	I
MOR-SCP-015	Tout écart au périmètre approuvé DOIT faire l'objet d'un ADR, d'une estimation et d'une décision explicite du Product Owner.	MUST	P0	I/T

5.2 Architecture et frontières

Séparation des responsabilités, contrôle/data plane et dépendances.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-ARC-001	Core, adaptateurs, workers, plugins et agents externes DOIVENT être séparés par des interfaces versionnées et documentées.	MUST	P0-P1	I/T
MOR-ARC-002	Le cœur NE DOIT charger aucune bibliothèque SALOME, KERNEL, CORBA, SALOMEDS, Qt, PyQt ou omniORB.	MUST	P0-P1	I/T
MOR-ARC-003	Aucun type OCCT, VTK, Kokkos, SALOME, STL ni exception C++ NE DOIT apparaître dans l'ABI publique C.	MUST	P0-P1	I/T
MOR-ARC-004	Les dépendances lourdes DOIVENT être optionnelles et découvertes par négociation de capacités.	MUST	P0-P1	I/T
MOR-ARC-005	Le DAG, les manifestes, les blocs CAS et les résultats publiés DOIVENT être immuables.	MUST	P0-P1	I/T
MOR-ARC-006	Les composants externes instables, copyleft, propriétaires ou non fiables DOIVENT pouvoir être exécutés hors processus.	MUST	P0-P1	I/T
MOR-ARC-007	Le plan de contrôle NE DOIT transporter que métadonnées, statuts, URI, hashes et diagnostics.	MUST	P0-P1	I/T

ID	Exigence normative	Prio.	Phase	Preuve
MOR-ARC-008	Les payloads massifs DOIVENT utiliser le data plane par CAS, fichiers, Arrow IPC ou ADIOS2 selon le contexte.	MUST	P0-P1	I/T
MOR-ARC-009	Aucune interface graphique NE DOIT être requise par le cœur, les tests, les workers ou les déploiements HPC.	MUST	P0-P1	I/T
MOR-ARC-010	Un monorepo principal DOIT regrouper le socle permissif jusqu'à P1 afin de limiter les coûts de CI et de coordination.	MUST	P0-P1	I/T
MOR-ARC-011	Les frontières de licence ou de runtime DOIVENT justifier les dépôts distincts morphoia-io-med et morphoia-salome-agent.	MUST	P0-P1	I/T
MOR-ARC-012	Les connecteurs dépendant d'ABI hôtes instables PEUVENT devenir des dépôts séparés à partir de P2.	MAY	P2+	I
MOR-ARC-013	Le système DOIT fonctionner en mode entièrement hors ligne, y compris validation, import, calcul CPU et consultation des résultats.	MUST	P0-P1	I/T
MOR-ARC-014	Le système NE DOIT supposer aucun état global caché susceptible de modifier la sémantique d'un appel.	MUST	P0-P1	I/T

5.3 Développement et toolchain

Langages, build, qualité du code et dépendances.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-DEV-001	Le cœur natif DOIT utiliser C++20.	MUST	P0-P1	T/I
MOR-DEV-002	Le contrat binaire public DOIT utiliser une ABI C11 versionnée.	MUST	P0-P1	T/I
MOR-DEV-003	La distribution de référence DOIT cibler Python 3.13 ; Python 3.12 DOIT rester compatible pendant P0-P1.	MUST	P0-P1	T/I
MOR-DEV-004	Python 3.14 NE DOIT être annoncé qu'après validation des builds VTK et de toute la pile native.	MUST	P0-P1	T/I
MOR-DEV-005	Le build natif DOIT utiliser CMake et des CMake Presets versionnés.	MUST	P0-P1	T/I
MOR-DEV-006	Spack DOIT constituer le chemin officiel de concrétisation des dépendances en environnement HPC.	MUST	P0-P1	T/I
MOR-DEV-007	Le code C++ DOIT être formaté automatiquement et analysé par une configuration clang-tidy versionnée.	MUST	P0-P1	T/I
MOR-DEV-008	Les nouveaux avertissements compilateur sur le code Morphoia DOIVENT bloquer la CI Tier 1.	MUST	P0-P1	T/I
MOR-DEV-009	Les exceptions C++ NE DOIVENT jamais franchir l'ABI C.	MUST	P0-P1	T/I
MOR-DEV-010	Toute mémoire possédée DOIT suivre RAII et les pointeurs bruts propriétaires sont interdits dans le nouveau code.	MUST	P0-P1	T/I
MOR-DEV-011	Le code Python DOIT être formaté, linté, typé et testé automatiquement.	MUST	P0-P1	T/I
MOR-DEV-012	Les dépendances DOIVENT être verrouillées ou concrétisées avec versions, options et hashes.	MUST	P0-P1	T/I
MOR-DEV-013	Toute dépendance vendorisée DOIT avoir provenance, licence, version et justification documentées.	MUST	P0-P1	T/I
MOR-DEV-014	Les secrets, données patient et gros binaires NE DOIVENT jamais être commités dans les dépôts publics.	MUST	P0-P1	T/I
MOR-DEV-015	Chaque module public DOIT posséder un CODEOWNER principal et un suppléant avant P2.	MUST	P0-P1	T/I
MOR-DEV-016	Le natif amont DOIT cibler OCCT 8.0.1 tandis que le runtime SALOME conserve sa pile patchée isolée.	MUST	P0-P1	T/I

5.4 API, ABI et versionnement

Contrats publics, mémoire, erreurs, stabilité et compatibilité.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-API-001	Les objets de l'ABI C DOIVENT utiliser des handles opaques.	MUST	P0-P1	T/I
MOR-API-002	Toute structure C extensible DOIT commencer par struct_size et abi_version.	MUST	P0-P1	T/I
MOR-API-003	Les chaînes DOIVENT être UTF-8 avec longueur explicite et sans hypothèse de terminaison implicite.	MUST	P0-P1	T/I
MOR-API-004	L'allocateur, la propriété, la durée de vie, le device et la synchronisation de toute vue mémoire DOIVENT être explicites.	MUST	P0-P1	T/I
MOR-API-005	La destruction d'un objet DOIT être fournie par le module qui l'a alloué.	MUST	P0-P1	T/I
MOR-API-006	La sûreté vis-à-vis des threads DOIT être documentée pour chaque famille d'API.	MUST	P0-P1	T/I
MOR-API-007	Les erreurs DOIVENT exposer code stable, sévérité, contexte, cause, éléments concernés et recommandation.	MUST	P0-P1	T/I
MOR-API-008	Tout module DOIT exposer une négociation de capacités et ses versions effectives.	MUST	P0-P1	T/I
MOR-API-009	L'API Python DOIT fournir des context managers et une libération déterministe des ressources.	MUST	P0-P1	T/I
MOR-API-010	La CLI DOIT fournir un rendu humain et un mode JSON machine-readable avec codes de sortie stables.	MUST	P0-P1	T/I
MOR-API-011	Toute opération longue DOIT exposer progression, timeout, annulation et état terminal.	MUST	P0-P1	T/I

ID	Exigence normative	Prio.	Phase	Preuve
MOR-API-012	Les requêtes rejouables DOIVENT accepter une clé d'idempotence.	MUST	P0-P1	T/I
MOR-API-013	Les dates DOIVENT être en UTC au format RFC 3339 et les durées dans des unités explicites.	MUST	P0-P1	T/I
MOR-API-014	Les paquets, API et protocoles DOIVENT suivre SemVer.	MUST	P0-P1	T/I
MOR-API-015	Les changements de schéma incompatibles DOIVENT inclure un outil de migration et des fixtures avant/après.	MUST	P0-P1	T/I
MOR-API-016	L'ABI C majeure v1 DOIT rester compatible pendant tout le cycle 1.x.	MUST	P4	T
MOR-API-017	Toute dépréciation publique DOIT rester annoncée pendant au moins deux versions mineures et six mois.	MUST	P4	I
MOR-API-018	Les extensions namespacées inconnues DOIVENT être préservées lors d'un round-trip qui ne les invalide pas.	MUST	P0-P1	T/I

6. Données, IR, CAS et formats

6.1 Morphoia IR

Identités, unités, repères, représentations, provenance et fidélité.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-IR-001	Le manifeste normatif DOIT utiliser JSON Schema 2020-12.	MUST	P0-P1	T
MOR-IR-002	La forme canonique signable DOIT suivre RFC 8785 ou une canonicalisation équivalente publiée et testée.	MUST	P0-P1	T
MOR-IR-003	Tout objet DOIT distinguer identité logique, révision et hash de contenu.	MUST	P0-P1	T
MOR-IR-004	Les identités logiques DEVRAIENT utiliser UUIDv7 conformément à RFC 9562.	MUST	P0-P1	T
MOR-IR-005	Toute référence binaire DOIT inclure digest SHA-256, taille, media type et URI résoluble.	MUST	P0-P1	T
MOR-IR-006	Les unités DOIVENT utiliser une notation non ambiguë compatible UCUM avec facteur vers le SI vérifiable.	MUST	P0-P1	T
MOR-IR-007	Tout repère DOIT déclarer dimension, origine, axes, handedness, ordre de matrice et unité.	MUST	P0-P1	T
MOR-IR-008	Toute transformation entre repères DOIT être conservée et ne jamais être appliquée silencieusement.	MUST	P0-P1	T
MOR-IR-009	Le bloc source DOIT contenir URI, licence, auteur ou détenteur, date, outil source et empreinte binaire.	MUST	P0-P1	T
MOR-IR-010	Le bloc produit DOIT conserver assemblages, instances, noms, couleurs, couches et propriétés disponibles.	MUST	P0-P1	T
MOR-IR-011	Les représentations B-Rep, mesh, points, voxels, volumes, champs, scènes, tenseurs et MVX DOIVENT rester typées séparément.	MUST	P0-P1	T
MOR-IR-012	La provenance DOIT former un DAG acyclique contenant outil, version, commit, paramètres, parents, environnement, graines et horodatages.	MUST	P0-P1	T
MOR-IR-013	Chaque événement de fidélité DOIT décrire propriété, valeur avant/après, métrique, seuil, gravité et décision.	MUST	P0-P1	T
MOR-IR-014	Le bloc sécurité DOIT déclarer classification, sensibilité, politique d'accès, signature et niveau de confiance.	MUST	P0-P1	T
MOR-IR-015	Les tableaux massifs et binaires NE DOIVENT pas être incorporés au manifeste JSON.	MUST	P0-P1	T
MOR-IR-016	Un identifiant topologique instable DOIT inclure la stratégie de correspondance et son niveau de confiance.	MUST	P0-P1	T
MOR-IR-017	Une sortie IA DOIT créer une branche révisable et NE DOIT jamais remplacer directement une autorité géométrique.	MUST	P0-P1	T
MOR-IR-018	Les schémas DOIVENT fournir exemples valides, invalides, cas limites et tests de migration.	MUST	P0-P1	T

6.2 Stockage adressé par contenu

Immutabilité, streaming, reprise, intégrité et rétention.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-CAS-001	Le CAS DOIT utiliser SHA-256 comme identité publiée.	MUST	P0-P1	T
MOR-CAS-002	BLAKE3 PEUT être utilisé pour le chunking interne mais NE DOIT pas créer une seconde identité publique.	MAY	P0-P1	I/T
MOR-CAS-003	La publication d'un bloc DOIT être atomique après vérification de sa taille et de son hash.	MUST	P0-P1	T
MOR-CAS-004	La lecture et l'écriture DOIVENT fonctionner en streaming.	MUST	P0-P1	T
MOR-CAS-005	Le CAS DOIT supporter des artefacts de plus de 2 Gio sans chargement intégral en mémoire.	MUST	P0-P1	T
MOR-CAS-006	Un cache local et au moins un backend POSIX ou objet distant DOIVENT partager les mêmes identités.	MUST	P0-P1	T

ID	Exigence normative	Prio.	Phase	Preuve
MOR-CAS-007	Le garbage collection NE DOIT jamais supprimer un bloc accessible depuis un manifeste épinglé.	MUST	P0-P1	T
MOR-CAS-008	Les corruptions de transfert DOIVENT être détectées avant exposition au consommateur.	MUST	P0-P1	T
MOR-CAS-009	Les reprises DOIVENT vérifier les chunks déjà validés et éviter leur retransfert.	MUST	P0-P1	T
MOR-CAS-010	Les URI externes DOIVENT être résolues par adaptateur et ne jamais servir de chemin arbitraire non validé.	MUST	P0-P1	T
MOR-CAS-011	Les quotas, politiques de rétention, pinning et purge DOIVENT être configurables par workspace.	MUST	P0-P1	T
MOR-CAS-012	Toute transformation DOIT produire un nouveau bloc et un nouveau nœud de provenance, sans modification in-place de l'entrée.	MUST	P0-P1	T

6.3 Profils d'entrée/sortie

Formats testés, niveaux A-D et interdiction des promesses universelles.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-IO-001	Chaque format DOIT avoir un profil de niveau A, B, C ou D et non une simple mention supporté.	MUST	P0-P2	T/I
MOR-IO-002	Chaque profil DOIT préciser lecture, écriture, attributs, unités, limites, pertes, tolérances et corpus.	MUST	P0-P2	T/I
MOR-IO-003	Les fichiers originaux DOIVENT être conservés avec hash et licence lorsqu'ils constituent l'autorité.	MUST	P0-P2	T/I
MOR-IO-004	Tout échec partiel DOIT produire un diagnostic structuré et la liste des entités concernées.	MUST	P0-P2	T/I
MOR-IO-005	Le profil STEP/AP242 initial DOIT couvrir unités, placements, assemblages, noms, couleurs, couches et géométrie B-Rep.	MUST	P0-P2	T/I
MOR-IO-006	Le profil STEP/AP242 initial NE DOIT pas revendiquer une couverture universelle de la PMI ou du topological naming.	MUST	P0-P2	T/I
MOR-IO-007	La tessellation NE DOIT jamais remplacer implicitement l'autorité B-Rep.	MUST	P0-P2	T/I
MOR-IO-008	Les tolérances géométriques DOIVENT être déclarées par actif de conformité.	MUST	P0-P2	T/I
MOR-IO-009	L'import d'un format propriétaire DOIT rester un plugin optionnel et produire le même rapport de fidélité.	MUST	P0-P2	T/I
MOR-IO-010	Aucun SDK CAO propriétaire NE DOIT être inclus dans la distribution OSS.	MUST	P0-P2	T/I
MOR-IO-011	Un import STEP tiers NE DOIT pas prétendre reconstruire l'historique paramétrique du logiciel source.	MUST	P0-P2	T/I
MOR-IO-012	Un maillage volumique DOIT conserver types de cellules, ordre, connectivité, groupes et orientation.	MUST	P0-P2	T/I
MOR-IO-013	Un champ DOIT déclarer support, composantes, unités, repère, nature intensive ou extensive et temps.	MUST	P0-P2	T/I
MOR-IO-014	LAS, LAZ et COPC DOIVENT conserver CRS, précision, classification et attributs inclus dans le profil.	MUST	P0-P2	T/I
MOR-IO-015	Un volume dense DOIT déclarer origine, spacing, direction, type, calibration et stratégie de chunking.	MUST	P0-P2	T/I
MOR-IO-016	Tout pipeline PDAL DOIT déclarer s'il est streamable ; un stage non streamable DOIT annoncer son coût de matérialisation.	MUST	P0-P2	T/I
MOR-IO-017	OpenVDB et NanoVDB DOIVENT rester des représentations spécialisées de volumes creux.	MUST	P2	T
MOR-IO-018	gITF, OpenUSD et MVX DOIVENT être marqués comme dérivés de présentation ou d'apprentissage, jamais comme autorité CAO.	MUST	P0-P2	T/I
MOR-IO-019	Toute réparation géométrique automatique DOIT être désactivable et produire un registre avant/après.	MUST	P0-P2	T/I

6.4 DICOM et confidentialité

Autorité médicale, Frame of Reference, désidentification et hors ligne.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-DIC-001	Les objets DICOM originaux et leur graphe de références DOIVENT rester l'autorité médicale.	MUST	P0-P1	T/I
MOR-DIC-002	Study, Series, SOP Instance UIDs et Frame of Reference DOIVENT être conservés ou remappés selon un profil explicite.	MUST	P0-P1	T/I
MOR-DIC-003	Orientation patient, origine, spacing et géométrie par frame DOIVENT être validés sur phantoms.	MUST	P0-P1	T/I
MOR-DIC-004	SEG, SR, objets RT et tags privés rencontrés DOIVENT être conservés, quarantainés ou supprimés selon une politique déclarée.	MUST	P0-P1	T/I
MOR-DIC-005	Une conversion NIFTI, VTKHDF, MED ou mesh NE DOIT jamais remplacer le graphe DICOM.	MUST	P0-P1	T/I
MOR-DIC-006	La désidentification DOIT référencer un profil DICOM PS3.15 et consigner chaque action.	MUST	P0-P1	T/I
MOR-DIC-007	Le remappage des UID DOIT rester cohérent dans tout le graphe.	MUST	P0-P1	T/I
MOR-DIC-008	La présence possible d'annotations brûlées dans les pixels DOIT être détectée ou signalée pour revue humaine.	MUST	P0-P1	T/I
MOR-DIC-009	Aucun identifiant patient NE DOIT apparaître dans logs, traces, noms de fichiers publics ou télémedie.	MUST	P0-P1	T/I

ID	Exigence normative	Prio.	Phase	Preuve
MOR-DIC-010	Les CI publiques DOIVENT utiliser exclusivement phantoms, données synthétiques ou jeux juridiquement validés.	MUST	P0-P1	T/I
MOR-DIC-011	Le moteur DOIT fonctionner sans télémétrie par défaut.	MUST	P0-P1	T/I
MOR-DIC-012	La classification, la durée de conservation et la politique d'accès DOIVENT être portées par le manifeste de sécurité.	MUST	P0-P1	T/I
MOR-DIC-013	Les traitements de données sensibles DOIVENT être auditable sans journaliser leur contenu.	MUST	P0-P1	T/I
MOR-DIC-014	Le logiciel DOIT fournir les éléments techniques nécessaires à une AIPD sans prétendre assurer à lui seul la conformité RGPD ou HDS.	MUST	P2	I

7. Visualisation, calcul, IA et MVX

7.1 Visualisation et streaming

VTK d'abord ; renderers et viewport spécialisés ensuite.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-VIS-001	Le MVP DOIT utiliser VTK headless et ParaView comme chemin principal de visualisation scientifique.	MUST	P1	T/I
MOR-VIS-002	ParaVis DOIT servir uniquement d'oracle SALOME et NE DOIT pas devenir un backend Morphoia.	MUST	P1	T/I
MOR-VIS-003	Les tessellations B-Rep DOIVENT être régénérables à partir de tolérances chordales et angulaires versionnées.	MUST	P1	T/I
MOR-VIS-004	Les points DOIVENT être streamés par COPC, EPT ou octree avec budget de points.	MUST	P1	T/I
MOR-VIS-005	Les volumes denses DOIVENT utiliser des bricks multirésolution et un transfert progressif.	MUST	P1	T/I
MOR-VIS-006	Le viewer NE DOIT pas charger par défaut l'intégralité d'un résultat HPC.	MUST	P1	T/I
MOR-VIS-007	ANARI PEUT fournir des renderers scientifiques interchangeables après le MVP.	MAY	P2	D
MOR-VIS-008	Vulkan PEUT fournir viewport, picking, culling et calcul proche du rendu mais NE DOIT pas servir de solveur général.	MAY	P2-P3	D
MOR-VIS-009	OpenUSD, Hydra et MaterialX PEUVENT servir à la composition et à la présentation dérivées.	MAY	P2-P3	D
MOR-VIS-010	Aucune application desktop Morphoia autonome NE DOIT être développée au MVP.	MUST	P1	T/I

7.2 Simulation et portabilité GPU

Workers ciblés, contrats de problème et politiques numériques.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-CMP-001	Un worker CPU OpenMP DOIT être disponible comme référence numérique.	MUST	P0-P2	T/A
MOR-CMP-002	Les workers CUDA et HIP DOIVENT être des builds distincts issus du même code source portable.	MUST	P0-P2	T/A
MOR-CMP-003	Chaque backend GPU DOIT produire un binaire ciblé ; aucun binaire GPU universel ne doit être promis.	MUST	P0-P2	T/A
MOR-CMP-004	Un build Kokkos NE DOIT activer qu'un backend device principal, conformément au worker ciblé.	MUST	P0-P2	T/A
MOR-CMP-005	CPU x86-64 et ARM64 DOIVENT constituer le Tier 1 de référence.	MUST	P0-P2	T/A
MOR-CMP-006	CUDA et HIP ou ROCm DOIVENT constituer les backends GPU Tier 1 après validation P0.	MUST	P0-P2	T/A
MOR-CMP-007	SYCL DOIT rester Tier 2 jusqu'à validation matérielle.	MUST	P3	T
MOR-CMP-008	OpenCL DOIT rester Tier 3 ou best effort.	SHOULD	P3+	I
MOR-CMP-009	Les capacités device, précision, mémoire et bibliothèques effectives DOIVENT être interrogables.	MUST	P0-P2	T/A
MOR-CMP-010	Les accélérations constructeur NE DOIVENT pas changer silencieusement la sémantique physique.	MUST	P0-P2	T/A
MOR-CMP-011	Un problem descriptor DOIT être indépendant du backend.	MUST	P0-P2	T/A
MOR-CMP-012	Un execution profile DOIT définir worker, précision, devices, MPI, partitionnement et politique mémoire.	MUST	P0-P2	T/A
MOR-CMP-013	Un result set DOIT contenir champs, scalaires, historique, résidus, logs, checkpoints, versions et métriques.	MUST	P0-P2	T/A
MOR-CMP-014	Les options constructeur DOIVENT être namespacées.	MUST	P0-P2	T/A
MOR-CMP-015	Les modes numériques reference, portable, performance et exploratory DOIVENT être distincts.	MUST	P0-P2	T/A
MOR-CMP-016	SMESH DOIT rester fournisseur externe de maillages et non solveur.	MUST	P0-P2	T/A
MOR-CMP-017	PETSc DOIT être intégré par adaptateur ; MFEM ou libCEED ne sera ajouté qu'après benchmark.	MUST	P2	T
MOR-CMP-018	SOFA, FEniCSx, OpenFOAM et autres solveurs DOIVENT rester des processus, jobs ou adaptateurs externes.	MUST	P0-P2	T/A

ID	Exigence normative	Prio.	Phase	Preuve
MOR-CMP-019	Un écart durable supérieur à 2 fois face à une référence constructeur sur un kernel critique DOIT déclencher une réévaluation ou une spécialisation.	MUST	P0-P2	T/A

7.3 Interopérabilité IA

DLPack, Arrow, provenance des modèles et absence de copie cachée.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-AI-001	Morphoia DOIT exposer des données et réintégrer des prédictions, sans réimplémenter un framework d'apprentissage profond.	MUST	P0-P1	T
MOR-AI-002	Les tenseurs denses DOIVENT être exposés via DLPack versionné.	MUST	P0-P1	T
MOR-AI-003	Les données colonnaires DOIVENT être exposées via Arrow C Data ou Device.	MUST	P0-P1	T
MOR-AI-004	Les graphes et meshes DOIVENT être exposés comme vues d'indices accompagnées de contrats de durée de vie.	MUST	P0-P1	T
MOR-AI-005	Un chemin éligible zéro copie DOIT conserver le pointeur de données et ne réaliser aucune allocation cachée.	MUST	P0-P1	T
MOR-AI-006	Toute copie ou synchronisation résiduelle DOIT être instrumentée et motivée.	MUST	P0-P1	T
MOR-AI-007	PyTorch et ONNX Runtime DOIVENT rester externes au cœur.	MUST	P0-P1	T
MOR-AI-008	Les providers ONNX Runtime DOIVENT être packagés par cible et négociés par capacités.	MUST	P0-P1	T
MOR-AI-009	Le provider AMD d'ONNX Runtime DOIT utiliser MIGraphX ; l'ancien provider ROCm NE DOIT pas être retenu comme référence.	MUST	P0-P1	T
MOR-AI-010	Le contrat Morphoia DOIT utiliser l'API C stable d'ONNX Runtime et exclure les API marquées expérimentales.	MUST	P0-P1	T
MOR-AI-011	CUDA, cuDNN, TensorRT et autres SDK constructeur NE DOIVENT pas être prérequis pour ouvrir ou traiter un dataset au niveau CPU.	MUST	P0-P1	T
MOR-AI-012	Une prédiction DOIT être un champ, une annotation, une segmentation, un score ou une proposition versionnée.	MUST	P0-P1	T
MOR-AI-013	Une prédiction DOIT conserver modèle, poids, commit, prétraitement, device, précision, graines et incertitude.	MUST	P0-P1	T
MOR-AI-014	Une prédiction DOIT créer une branche révisable et NE DOIT pas modifier l'autorité.	MUST	P0-P1	T
MOR-AI-015	Les modèles persistants d'inférence DEVRAIENT utiliser ONNX accompagné d'une fiche de modèle.	MUST	P0-P1	T
MOR-AI-016	Les échanges inter-processus DOIVENT utiliser Arrow IPC, mémoire partagée ou ADIOS2 et non des copies JSON.	MUST	P0-P1	T
MOR-AI-017	DLPack et Arrow C Device NE DOIVENT pas être présentés comme des contrats inter-processus.	MUST	P0-P1	T

7.4 Expérimentation MVX

Validation progressive indépendante du succès du socle.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-MVX-001	MVX DOIT rester une famille de représentations dérivées expérimentales.	MUST	P0-P1	T/A
MOR-MVX-002	Le premier profil DOIT couvrir LP3 selon les projections X, Y et Z.	MUST	P0-P1	T/A
MOR-MVX-003	Chaque encodage DOIT référencer l'objet source, les vues, la résolution, les paramètres et la provenance.	MUST	P0-P1	T/A
MOR-MVX-004	L'objet source DOIT être normalisé selon une procédure déterministe enregistrant centre, échelle, axes et orientation.	MUST	P0-P1	T/A
MOR-MVX-005	Le trajet objet 3D vers MVX vers reconstruction DOIT être validé sur 30 à 50 objets avant le corpus d'environ 1 000.	MUST	P0-P1	T/A
MOR-MVX-006	Le corpus d'environ 1 000 objets DOIT être stratifié par topologie, complexité, symétrie, occlusion et source.	MUST	P0-P1	T/A
MOR-MVX-007	Les splits DOIVENT être faits par identité d'objet et empêcher tout duplicat ou variante proche entre entraînement et test.	MUST	P0-P1	T/A
MOR-MVX-008	Les métriques DOIVENT inclure Chamfer normalisé, Hausdorff, F-score, IoU voxel ou SDF, normales, topologie et score multi-vues.	MUST	P0-P1	T/A
MOR-MVX-009	Les taux de décodage réussi, watertightness, manifoldness et auto-intersections DOIVENT être publiés.	MUST	P0-P1	T/A
MOR-MVX-010	Le coût de calcul, stockage, transfert et reconstruction DOIT être mesuré.	MUST	P0-P1	T/A
MOR-MVX-011	MVX DOIT être comparé à au moins une représentation de surface et une représentation volumique à budget de stockage comparable.	MUST	P0-P1	T/A
MOR-MVX-012	Les seuils définitifs DOIVENT être gelés après le micro-corpus, avant l'exécution des 1 000 objets.	MUST	P0-P1	T/A
MOR-MVX-013	La cible initiale de faisabilité DEVRAIT atteindre au minimum 95 % de décodages valides et une médiane Chamfer normalisée inférieure ou égale à 1e-3.	SHOULD	P0-P1	T/A

ID	Exigence normative	Prio.	Phase	Preuve
MOR-MVX-014	La cible initiale DEVRAIT atteindre un P95 Chamfer normalisé inférieur ou égal à 5e-3 et une cohérence médiane des normales supérieure ou égale à 0,95.	SHOULD	P0-P1	T/A
MOR-MVX-015	Les seuils indicatifs PEUVENT être révisés une seule fois en P0 sur preuve publiée, avant gel du protocole.	MUST	P0-P1	T/A
MOR-MVX-016	Cent pour cent des échecs DOIVENT être catégorisés et reliés aux artefacts source, encodage et reconstruction.	MUST	P0-P1	T/A
MOR-MVX-017	Un échec MVX DOIT bloquer le passage au JEPA, sans bloquer le socle d'interopérabilité Morphoia.	MUST	P0-P1	T/A
MOR-MVX-018	Aucun entraînement JEPA sur le corpus complet NE DOIT commencer avant validation de la reconstruction et de la reproductibilité MVX.	MUST	P0-P1	T/A

8. Extensions, SALOME et HPC

8.1 Plugins et connecteurs

Quatre niveaux d'extension et compatibilité par capacités.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-PLG-001	Les niveaux d'extension DOIVENT couvrir ABI C native, Python, processus isolé et WASI.	MUST	P1-P2	T/I
MOR-PLG-002	Le SDK natif DOIT utiliser exclusivement l'ABI C versionnée.	MUST	P1-P2	T/I
MOR-PLG-003	Tout plugin DOIT fournir manifeste, version, ABI, capacités, licences SPDX, architectures, permissions et limites.	MUST	P1-P2	T/I
MOR-PLG-004	La compatibilité DOIT être négociée par capacités et non par nom de plugin.	MUST	P1-P2	T/I
MOR-PLG-005	La signature, l'origine et l'état de confiance DOIVENT être exposés avant chargement.	MUST	P1-P2	T/I
MOR-PLG-006	Les plugins de parsing non fiables DOIVENT être isolables hors processus.	MUST	P1-P2	T/I
MOR-PLG-007	Les plugins Python DOIVENT être chargés depuis un environnement contrôlé.	MUST	P1-P2	T/I
MOR-PLG-008	Les adaptateurs aux ABI hôtes instables DOIVENT publier une matrice de versions.	MUST	P1-P2	T/I
MOR-PLG-009	Morphoia NE DOIT pas promettre le chargement binaire direct des plugins Blender, Godot, ParaView ou Hydra.	MUST	P1-P2	T/I
MOR-PLG-010	Les priorités DOIVENT être SALOME P0-P1, ParaView/FreeCAD/Slicer P1-P2, OpenUSD P2, Blender/Godot P3.	MUST	P0-P3	I

8.2 Profil SALOME 9.16

Backend isolé, artefacts standards et oracle différentiel.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-SAL-001	SALOME 9.16 DOIT être traité comme backend CAE optionnel et oracle différentiel dès P0.	MUST	P0-P2	T/I
MOR-SAL-002	morphoia-core NE DOIT charger aucune dépendance SALOME, Qt, PyQt ou omniORB.	MUST	P0-P2	T/I
MOR-SAL-003	MEDCoupling et MEDLoader PEUVENT être liés uniquement dans un paquet morphoia-io-med LGPL dynamique séparé.	MUST	P0-P2	T/I
MOR-SAL-004	SHAPER, GEOM et SMESH DOIVENT s'exécuter dans un agent SALOME hors processus.	MUST	P0-P2	T/I
MOR-SAL-005	Aucun TopoDS_Shape, objet CORBA, handle SALOME ou pointeur NE DOIT franchir la frontière.	MUST	P0-P2	T/I
MOR-SAL-006	STEP/AP242 DOIT être la frontière CAO générale.	MUST	P0-P2	T/I
MOR-SAL-007	XAO DOIT être le pont spécialisé forme, topologie, groupes et champs.	MUST	P0-P2	T/I
MOR-SAL-008	MED DOIT transporter maillages et champs.	MUST	P0-P2	T/I
MOR-SAL-009	Les études HDF DOIVENT être conservées comme artefacts opaques de reproductibilité.	MUST	P0-P2	T/I
MOR-SAL-010	Tout artefact massif DOIT passer par URI, hash et taille, jamais dans JSON, CORBA ou gRPC.	MUST	P0-P2	T/I
MOR-SAL-011	Le profil OSS de référence NE DOIT contenir aucun MeshGems.	MUST	P0-P2	T/I
MOR-SAL-012	L'agent DOIT être lancé sans GUI, même si l'installation SALOME contient Qt ou PyQt.	MUST	P0-P2	T/I
MOR-SAL-013	ParaVis DOIT être un oracle et ParaView ou VTK amont le chemin principal.	MUST	P0-P2	T/I
MOR-SAL-014	Une divergence SALOME DOIT déclencher une analyse et SALOME NE DOIT pas être déclaré automatiquement vérité terrain.	MUST	P0-P2	T/I
MOR-SAL-015	Le protocole DOIT exposer ProbeCapabilities, Submit, Observe, Cancel, Publish et Health.	MUST	P0-P2	T/I
MOR-SAL-016	ProbeCapabilities DOIT publier versions, modules, formats MED, maillages OSS, limites et licences.	MUST	P0-P2	T/I
MOR-SAL-017	Submit DOIT accepter opération, paramètres, entrées URI/hash, budgets et sorties attendues.	MUST	P0-P2	T/I
MOR-SAL-018	Observe et Cancel DOIVENT fournir état, progression, logs, timeout et annulation idempotente.	MUST	P0-P2	T/I

ID	Exigence normative	Prio.	Phase	Preuve
MOR-SAL-019	Publish DOIT produire manifestes STEP, XAO, MED ou HDF, tailles, hashes, unités, pertes et métriques.	MUST	P0-P2	T/I
MOR-SAL-020	Chaque tentative DOIT produire environnement, logs et état terminal explicite.	MUST	P0-P2	T/I
MOR-SAL-021	external_refs.salome DOIT conserver runtime, module, étude, groupes, algorithmes, numérotation, champs et artefacts de replay.	MUST	P0-P2	T/I
MOR-SAL-022	Un study_entry SALOME NE DOIT jamais servir d'UUID Morphoia.	MUST	P0-P2	T/I
MOR-SAL-023	Le profil d'export MED effectif DOIT être explicite et testé intervention.	MUST	P0-P2	T/I
MOR-SAL-024	Le manifeste runtime SALOME DOIT enregistrer Python 3.9.14, C++14, OCCT 7.9.0 patché, MED 6.0.1 et les versions réellement détectées.	MUST	P0-P2	T/I
MOR-SAL-025	L'overhead chaud du bridge DOIT rester inférieur à 15 % sur les opérations longues du benchmark convenu.	MUST	P0-P2	T/I
MOR-SAL-026	YACS et JOBMANAGER NE DOIVENT être développés qu'après validation d'un cas utilisateur réel.	MUST	P2	I
MOR-SAL-027	Une image SALOME redistribuée DOIT être distincte et accompagnée de SBOM, notices, sources et patches exigibles.	MUST	P1-P2	I

8.3 Exécution HPC

Bundles immuables, profils de site, Slurm, Spack et Apptainer.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-HPC-001	Un job bundle DOIT être immuable, autosuffisant, signé et validé par schéma.	MUST	P0-P3	T/D
MOR-HPC-002	Le worker HPC DOIT être headless, stateless et exécutable sans serveur central.	MUST	P0-P3	T/D
MOR-HPC-003	Le job bundle DOIT référencer entrées, hashes, worker, ressources, environnement, graines, sorties et checkpoints.	MUST	P0-P3	T/D
MOR-HPC-004	Slurm DOIT être l'ordonnanceur de référence ; Morphoia NE DOIT pas réimplémenter un scheduler.	MUST	P0-P3	T/D
MOR-HPC-005	Le MPI, les drivers et les bibliothèques proches du matériel DOIVENT être ceux du centre selon le profil de site.	MUST	P0-P3	T/D
MOR-HPC-006	Les environnements HPC DOIVENT être concrétisés avec Spack ou un équivalent documenté et verrouillé.	MUST	P0-P3	T/D
MOR-HPC-007	Le profil Spack DOIT archiver spack.yaml, spack.lock, version Spack, commit du dépôt de packages, compilateurs et externals.	MUST	P0-P3	T/D
MOR-HPC-008	Les images DOIVENT être compatibles Apptainer et ne pas embarquer arbitrairement drivers ou MPI incompatibles.	MUST	P0-P3	T/D
MOR-HPC-009	Chaque image Apptainer DOIT être testée avec le MPI, PMI ou PMIX, les drivers et l'interconnexion du site.	MUST	P0-P3	T/D
MOR-HPC-010	Toute opération longue DOIT pouvoir publier un checkpoint et reprendre sans recalcul des résultats validés.	MUST	P0-P3	T/D
MOR-HPC-011	Les profils de site DOIVENT déclarer scheduler, architecture, modules, I/O, quotas, réseau et politique de purge.	MUST	P0-P3	T/D
MOR-HPC-012	Le DAG Morphoia et les bundles Slurm DOIVENT rester autoritaires devant YACS ou JOBMANAGER.	MUST	P0-P3	T/D
MOR-HPC-013	Les caches DOIVENT accepter POSIX, S3 compatible ou stockage du centre avec hashes identiques.	MUST	P0-P3	T/D
MOR-HPC-014	La bêta DOIT être validée sur au moins un environnement NVIDIA et un environnement AMD.	MUST	P3	D
MOR-HPC-015	LUMI ou équivalent HIP et Leonardo ou équivalent CUDA DOIVENT être ciblés en priorité.	MUST	P3	D
MOR-HPC-016	JUPITER ARM64/CUDA DOIT être testé si l'accès est disponible.	SHOULD	P3	D

9. Sécurité, performance, qualité et gouvernance

9.1 Sécurité et supply chain

Entrées non fiables, isolation, SBOM, signature et données sensibles.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-SEC-001	Un modèle de menace DOIT couvrir fichiers hostiles, plugins, agents, CAS, workers HPC et chaîne de build.	MUST	P0-P2	T/I
MOR-SEC-002	Chaque parseur DOIT appliquer limites de taille, profondeur, nombre d'entités, temps et mémoire.	MUST	P0-P2	T/I
MOR-SEC-003	Les chemins, archives, liens symboliques et URI DOIVENT être protégés contre traversée et écrasement.	MUST	P0-P2	T/I
MOR-SEC-004	Les parseurs critiques DOIVENT disposer de harnesses de fuzzing sous sanitizers.	MUST	P0-P2	T/I
MOR-SEC-005	Le réseau DOIT être désactivé par défaut pour les plugins et agents non autorisés.	MUST	P0-P2	T/I

ID	Exigence normative	Prio.	Phase	Preuve
MOR-SEC-006	Les processus isolés DOIVENT utiliser workspace temporaire, quotas et droits minimaux.	MUST	P0-P2	T/I
MOR-SEC-007	Chaque release DOIT inclure une SBOM SPDX ou CycloneDX.	MUST	P0-P2	T/I
MOR-SEC-008	La SBOM DEVRAIT cibler SPDX 3.0.1 ou CycloneDX 1.7 et satisfaire les Minimum Elements CISA 2026.	SHOULD	P0-P2	I
MOR-SEC-009	Les artefacts de release DOIVENT être signés et accompagnés de hashes et provenance de build.	MUST	P0-P2	T/I
MOR-SEC-010	La provenance de build DEVRAIT suivre SLSA 1.2 et les attestations in-toto Statement v1.	SHOULD	P0-P2	I
MOR-SEC-011	Les secrets DOIVENT être détectés automatiquement avant fusion.	MUST	P0-P2	T/I
MOR-SEC-012	Les dépendances DOIVENT faire l'objet d'un scan de vulnérabilités à chaque release.	MUST	P0-P2	T/I
MOR-SEC-013	Aucun défaut critique exploitable connu NE DOIT subsister dans une release acceptée.	MUST	P0-P2	T/I
MOR-SEC-014	Toute dérogation de vulnérabilité élevée DOIT être approuvée, motivée et datée d'expiration.	MUST	P0-P2	T/I
MOR-SEC-015	Les connexions distantes DOIVENT utiliser authentification mutuelle et chiffrement moderne configurables.	MUST	P2	T
MOR-SEC-016	Un SECURITY.md et un canal de divulgation privée DOIVENT être publiés avant v0.1.	MUST	P0-P2	T/I
MOR-SEC-017	Les job bundles NE DOIVENT contenir aucun secret en clair ; ils référencent des identités injectées par le site.	MUST	P0-P2	T/I
MOR-SEC-018	Les journaux DOIVENT exclure PHI, secrets, tokens et chemins sensibles.	MUST	P0-P2	T/I
MOR-SEC-019	Les sources DICOM identifiantes DOIVENT être séparées des dérivés et chiffrées au repos lorsque requis.	MUST	P0-P2	T/I
MOR-SEC-020	Tout export cloud ou HPC de données sensibles DOIT vérifier classification et destination autorisée.	MUST	P0-P2	T/I

9.2 Déterminisme et performance

Mesures bout-en-bout, SLO et politiques numériques.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-PER-001	Les modes reference, portable, performance et exploratory DOIVENT être explicitement distingués.	MUST	P0-P3	T/A
MOR-PER-002	Le mode reference DOIT figer versions, graines, précision, ordre et options numériques.	MUST	P0-P3	T/A
MOR-PER-003	Sur même matériel et environnement, métadonnées, topologies et données entières DOIVENT être identiques entre deux exécutions de référence.	MUST	P0-P3	T/A
MOR-PER-004	Les flottants inter-backends DOIVENT respecter des tolérances définies par grandeur et cas de conformité.	MUST	P0-P3	T/A
MOR-PER-005	fast-math, FMA, mixed precision et autotuning DOIVENT être enregistrés.	MUST	P0-P3	T/A
MOR-PER-006	Le matériel, OS, drivers, versions et paramètres de chaque benchmark DOIVENT être publiés.	MUST	P0-P3	T/A
MOR-PER-007	Les résultats DOIVENT séparer exécution froide, chaude, I/O, calcul, transfert et synchronisation.	MUST	P0-P3	T/A
MOR-PER-008	Les métriques DOIVENT inclure médiane, dispersion, nombre de répétitions et données brutes.	MUST	P0-P3	T/A
MOR-PER-009	Une régression médiane supérieure à 10 % sur un benchmark stable DOIT bloquer la release ou être approuvée.	MUST	P0-P3	T/A
MOR-PER-010	Le pipeline gros artefact DOIT traiter au moins un cas supérieur à 2 Gio avec une RAM inférieure à 50 % de sa taille.	MUST	P0-P3	T/A
MOR-PER-011	Les payloads du plan de contrôle DOIVENT rester inférieurs à 16 Mio par défaut ; les artefacts dépassant 64 Kio DEVRAIENT être externes.	MUST	P0-P3	T/A
MOR-PER-012	Les benchmarks DOIVENT mesurer temps au premier objet, temps total, débit, RSS, VRAM, taille des dérivés et cache hit.	MUST	P0-P3	T/A
MOR-PER-013	Le chargement d'un manifeste de 10 000 nœuds DEVRAIT rester inférieur à 2 secondes sur la machine de référence P0.	SHOULD	P0-P3	T
MOR-PER-014	Le chargement d'un manifeste de 100 000 nœuds DEVRAIT rester inférieur à 10 secondes et 4 Gio de RSS.	SHOULD	P0-P3	T
MOR-PER-015	Les seuils de scaling HPC définitifs DOIVENT être baselinés en P0-P1 avant de devenir contractuels en P3.	MUST	P0-P3	T/A
MOR-PER-016	Le hashing CAS DEVRAIT atteindre au moins 60 % du débit de sha256sum sur le même support et la même machine.	SHOULD	P0-P3	T

9.3 QA, CI et recette

Stratégie de tests, corpus, couverture et preuves.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-QA-001	La stratégie de tests DOIT couvrir unitaires, propriété, intégration, end-to-end, golden, différentiel, fuzz et performance.	MUST	P0-P4	T//A/D

ID	Exigence normative	Prio.	Phase	Preuve
MOR-QA-002	Le code Morphoia DOIT atteindre au minimum 85 % de couverture lignes et 75 % de couverture branches.	MUST	P0-P4	T/I/A/D
MOR-QA-003	IR, CAS, ABI et protocoles DOIVENT atteindre au minimum 90 % de couverture lignes et 85 % de couverture branches.	MUST	P0-P4	T/I/A/D
MOR-QA-004	Toute correction de défaut DOIT inclure un test de non-régression.	MUST	P0-P4	T/I/A/D
MOR-QA-005	Aucun test flaky connu NE DOIT rester bloquant sans propriétaire et date de correction.	MUST	P0-P4	T/I/A/D
MOR-QA-006	Toute release candidate DOIT cumuler au moins 72 heures-CPU de fuzzing sous sanitizer sans crash reproductible.	MUST	P0-P4	T/I/A/D
MOR-QA-007	Linux x86-64 CPU DOIT être testé à chaque pull request.	MUST	P0-P4	T/I/A/D
MOR-QA-008	CUDA, HIP, ARM64, SALOME et tests longs DOIVENT être exécutés nightly ou sur runners jalonnés.	MUST	P0-P4	T/I/A/D
MOR-QA-009	Windows CPU et SYCL DOIVENT être Tier 2 au plus tard en P3.	MUST	P3	T
MOR-QA-010	Les branches protégées DOIVENT exiger contrôles obligatoires et revue indépendante.	MUST	P0-P4	T/I/A/D
MOR-QA-011	Les changements d'ABI, schéma, licence ou sécurité DOIVENT exiger une revue CODEOWNER dédiée.	MUST	P0-P4	T/I/A/D
MOR-QA-012	Toute release DOIT être restructurable depuis un tag, un lockfile et une documentation publique.	MUST	P0-P4	T/I/A/D
MOR-QA-013	Le corpus minimal DOIT inclure 20 graphes IR, 20 STEP, 20 SMESH, 10 MED et 3 artefacts supérieurs à 2 Gio.	MUST	P0-P4	T/I/A/D
MOR-QA-014	Le corpus P1 DOIT viser 100 cas CAO, 100 DICOM et 100 cas points ou meshes redistribuables ou synthétiques.	MUST	P0-P4	T/I/A/D
MOR-QA-015	Chaque actif de conformité DOIT porter source, licence, hash, profil, invariants, tolérances et statut de redistribution.	MUST	P0-P4	T/I/A/D
MOR-QA-016	La conformité SALOME DOIT comparer unités, placements, topologie, groupes, champs, séries temporelles et structure ParaView/ParaVis.	MUST	P0-P4	T/I/A/D
MOR-QA-017	Cent pour cent des exigences MUST d'une tranche DOIVENT avoir une preuve reliée à un test, une inspection, une analyse ou une démonstration.	MUST	P0-P4	T/I/A/D
MOR-QA-018	L'acceptation d'une phase exige zéro anomalie Sev 1 ou Sev 2 ; toute Sev 3 résiduelle DOIT avoir propriétaire et échéance.	MUST	P0-P4	T/I/A/D

9.4 Licences

Distribution réellement open source et obligations par artefact.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-LIC-001	Le nouveau code, les schémas, les spécifications et les exemples DOIVENT être sous Apache-2.0 OR MIT.	MUST	P0-P1	I/T
MOR-LIC-002	La documentation originale DOIT être sous CC BY 4.0 sauf exception indiquée.	MUST	P0-P1	I/T
MOR-LIC-003	Les jeux de test créés DEVRAIENT être sous CC0 ou CC BY 4.0.	MUST	P0-P1	I/T
MOR-LIC-004	Chaque fichier source DOIT porter un identifiant SPDX approprié.	MUST	P0-P1	I/T
MOR-LIC-005	Chaque dépendance DOIT apparaître dans une matrice version, licence, liaison, redistribution et obligations.	MUST	P0-P1	I/T
MOR-LIC-006	Les données DOIVENT conserver une licence par actif ; aucune licence globale ne doit écraser celle des sources.	MUST	P0-P1	I/T
MOR-LIC-007	Les poids de modèles DOIVENT posséder leur propre licence compatible avec les données.	MUST	P0-P1	I/T
MOR-LIC-008	Le logo et la marque Morphoia DOIVENT être gérés séparément des licences de code.	MUST	P0-P1	I/T
MOR-LIC-009	Aucune clause non commerciale NE DOIT être ajoutée à une licence annoncée open source.	MUST	P0-P1	I/T
MOR-LIC-010	La séparation en processus NE DOIT jamais être présentée comme une exemption aux obligations de licence.	MUST	P0-P1	I/T
MOR-LIC-011	Aucun composant non redistribuable NE DOIT devenir obligatoire pour le chemin CPU de référence.	MUST	P0-P1	I/T
MOR-LIC-012	Toute release DOIT auditer l'artefact réellement distribué, options, codecs et plugins compris.	MUST	P0-P1	I/T
MOR-LIC-013	Le profil OpenUSD strictement open source DOIT exclure le JSON checker non conforme et utiliser un LicenseRef TOST-1.0 dans la SBOM.	MUST	P0-P1	I/T

9.5 Gouvernance

Rôles publics, ADR, DCO, mainteneurs et indépendance de la conformité.

ID	Exigence normative	Prio.	Phase	Preuve
MOR-GOV-001	Le projet DOIT publier gouvernance, rôles, processus de décision et politique de release.	MUST	P0-P2	I
MOR-GOV-002	Toute décision structurante DOIT être consignée dans un ADR.	MUST	P0-P2	I
MOR-GOV-003	Les changements majeurs DOIVENT suivre un RFC public avec délai de revue.	MUST	P2+	I

ID	Exigence normative	Prio.	Phase	Preuve
MOR-GOV-004	Les contributions DOIVENT utiliser un DCO plutôt qu'un CLA lourd, sauf contrainte juridique contraire.	MUST	P0-P2	I
MOR-GOV-005	Le projet DOIT publier Code of Conduct, CONTRIBUTING, SUPPORT et SECURITY.	MUST	P0-P2	I
MOR-GOV-006	Roadmap, backlog, critères de release et comptes rendus techniques DOIVENT être publics.	MUST	P0-P2	I
MOR-GOV-007	Aucun composant essentiel NE DOIT être privé.	MUST	P0-P2	I
MOR-GOV-008	Les domaines géométrie, médical, GPU/HPC, sécurité et licences DOIVENT avoir un mainteneur identifié.	MUST	P0-P2	I
MOR-GOV-009	La suite de conformité DOIT rester indépendante de l'implémentation afin de permettre d'autres moteurs compatibles.	MUST	P0-P2	I
MOR-GOV-010	Le Product Owner et autorité de domaine DOIT être Dr Olivier Ami jusqu'à décision de gouvernance contraire.	MUST	P0-P2	I

04

Contrats exécutables

Schémas, ABI, protocoles, diagnostics et profils de données

10. Morphoia IR v0.1

L'IR est une enveloppe sémantique et un graphe de provenance. Elle référence les payloads dans leurs formats adaptés. Le schéma racine ci-dessous est une baseline minimale à formaliser en JSON Schema pendant la première semaine de PO.

```
{
  schema_uri, schema_version,
  object_id, revision_id, created_at,
  authority, sources[], coordinate_frames[],
  representations[], fields[],
  graph: {nodes[], edges[], provenance[],
  fidelity_events[], external_refs: {},
  security: {}, signatures[], extensions: {}
}
```

10.1 Identité et canonicalisation

Concept	Choix	Règle
Identité logique	UUIDv7 recommandé	Survit aux nouvelles représentations et révisions.
Révision	Identifiant immuable	Toute modification crée une nouvelle révision.
Identité de contenu	sha256:	Calculée sur les octets ; vérifiée avant publication.
JSON canonique	RFC 8785/JCS	Utilisé pour hash, signature et égalité de manifeste.
Date	UTC RFC 3339	Aucune heure locale ambiguë dans le contrat.
Unités	UCUM + facteur SI	Validation dimensionnelle obligatoire.

10.2 Événements de fidélité

```
{
  event_id, operation_id, affected_entities[], property,
  before, after, metric, value, threshold,
  severity: info | warning | error | blocking,
  reversible, repair_id, decision, evidence_uri
}
```

10.3 Bloc SALOME

```
external_refs.salome = {
  salome_version, modules, python, os_image, manifest_hash,
  module, study, study_entry,
  mesh_algorithm, hypotheses[], groups[], geometry_bindings[],
  numbering, field_support, components, units, nature, time,
  losses[], repairs[], replay_artifacts[]
}
```

11. ABI C et SDK

L'ABI C protège Morphoia de la fragilité binaire des bibliothèques C++. Elle ne cherche pas à exposer tous les objets internes : elle expose des capacités stables, des handles et des vues mémoire à durée de vie explicite.

```
typedef struct morphoia_context_t morphoia_context_t;
typedef struct morphoia_job_t morphoia_job_t;

typedef struct {
  uint32_t struct_size;
  uint32_t abi_version;
  const char* data;
  size_t length;
} morphoia_string_view_t;

morphoia_status_t morphoia_context_create(
  const morphoia_context_options_t*, morphoia_context_t**);
```

```
void morphoia_context_destroy(morphoia_context_t*);
```

11.1 Taxonomie des diagnostics

Famille	Exemples	Action attendue
MOR-I/O	format, version, parsing, codec	Échec localisé ; entités et offset si disponibles.
MOR-GEOM	tolérance, invalidité, réparation	Métrique et événement de fidélité.
MOR-UNIT/FRAME	dimension, unité, repère	Bloquant si ambigu ou conversion silencieuse.
MOR-CAS	hash, taille, URI, quota	Aucune publication partielle.
MOR-DEVICE	backend, mémoire, sync	Fallback explicite ou refus.
MOR-SEC	permission, signature, donnée sensible	Refus par défaut et audit.
MOR-EXT	ABI, capacité, runtime	Diagnostics de compatibilité.

12. CLI et API Python

```
morphoia inspect ASSET --json
morphoia validate MANIFEST --profile step-ap242-m0
morphoia import INPUT --as step --workspace WORK
morphoia derive OBJECT --to vtkhdf --policy reference
morphoia run JOB.yaml --site local-cpu
morphoia salome probe --runtime salome-9.16
morphoia compare RESULT_A RESULT_B --invariants PROFILE
```

```
from morphoia import Workspace
```

```
with Workspace.open('work') as ws:
    obj = ws.import_asset('part.step', profile='step-ap242-m0')
    mesh = obj.derive('mesh', tolerance='0.05 mm')
    tensor = mesh.to_dlpack(device='cuda:0')
    result = ws.run('quality-summary', inputs=[mesh])
```

13. Job bundle et profil de site

```
job_id: uuid
schema_version: 0.1.0
inputs: [{uri, sha256, size}]
operation: morphoia.compute.kernel
worker: {name, version, backend}
resources: {nodes, tasks, cpus, gpus, memory, walltime}
numerical_policy: reference
environment_lock: spack.lock
outputs: [{role, destination, media_type}]
checkpoints: {interval, destination, resume_from}
provenance: {git_commit, image_digest, seeds}
```

13.1 États d'exécution

État	Terminal	Rejouable	Sémantique
accepted	Non	Oui	Requête validée et idempotency key réservée.
running	Non	Oui	Progression, logs et checkpoint possibles.
succeeded	Oui	Oui	Toutes les sorties sont publiées et hashées.
failed	Oui	Oui	Diagnostics structuré et sorties partielles non autoritaires.
cancelled	Oui	Oui	Annulation confirmée ; checkpoint conservé selon politique.
lost	Non	Oui	Lease expirée ; observation et reprise nécessaires.

14. Profils de formats initiaux

Profil	Entrée/sortie	Périmètre P1	Réserve
step-ap242-m0	R/W	B-Rep, unités, placements, assemblages, noms, couleurs, couches	PMI avancée et topological naming hors promesse
vtkhdf-field-m0	R/W	Meshes, cellules, champs points/cellules, temps simple	Options parallèles qualifiées par corpus
dicom-image-m0	R	Objets image, UUIDs, FoR, géométrie, pixels selon codecs disponibles	Écriture clinique non promise
dicom-seg-m0	R	SEG et références vers source	Profil partiel P1, enrichi P2
las-laz-m0	R/W	CRS, positions, couleur, intensité, classification	Extra dimensions par profil
copc-m0	R	Hiérarchie, requêtes spatiales, LOD	Écriture optionnelle
med-field-m0	R/W	Meshes, groupes, champs nœuds/cellules/Gauss et temps	Version MED explicitement verrouillée
mvx-lp3-m0	R/W	Trois projections X/Y/Z et paramètres de reconstruction	Expérimental, jamais autorisé

05

Réalisation

Lots, jalons, équipe, coûts et transfert

15. Lots de travaux

Lot	Objet	Livrables majeurs	Effort v1
WP0	Gouvernance, exigences, architecture, licences	Baseline, ADR, RACI, risques, SBOM initiale	4 pm
WP1	Core, ABI C, IR et CAS	Bibliothèques, schémas, migrations, Python/CLI	10 pm
WP2	CAO, meshes, points, formats scientifiques	STEP/AP242, OCCT/XDE, VTKHDF, LAS/LAZ/COPC	8 pm
WP3	DICOM et données médicales	Graphe, géométrie, profils de confidentialité, phantoms	6 pm
WP4	Compute, GPU, IA et MVX	Workers, Kokkos, DLPack/Arrow, protocole MVX	8 pm
WP5	SALOME, MED et conformité	Agent, io-med, SMESH/MED/XAO, oracle différentiel	7 pm
WP6	Visualisation et connecteurs	VTK/ParaView puis Slicer/FreeCAD	5 pm
WP7	HPC et reproductibilité	Bundles, Slurm, Spack, Apptainer, checkpoints	5 pm
WP8	QA, sécurité et releases	CI, fuzzing, benchmarks, signature, audit	5 pm
WP9	Documentation et transfert	Guides, API, tutoriels, formation	2 pm

Le total cible est d'environ 60 personnes-mois. Les lots ne sont pas séquentiels : WP0 et WP8 traversent toutes les phases ; WP1 constitue le chemin critique ; WP5 démarre dès P0.

15.1 Dépendances entre lots

Lot	Dépend de	Bloque
WP1	WP0 baseline	Tous les adaptateurs et workers
WP2	IR/CAS minimal	Vertical slice CAO/points
WP3	IR frames/security	Vertical slice médical
WP4	ABI memory + CAS	MVX, GPU, IA
WP5	Protocole + profils STEP/MED	Conformité SALOME
WP7	Worker + bundle schema	Bêta EuroHPC
WP8	Tous	Chaque gate et release

16. Plan P0 - six semaines

Semaine	Objectif	Sorties obligatoires
S0	Dépôts, licences et gouvernance	DCO, CODEOWNERS, ADR-001 à 014, CI CPU, matrice dépendances
S1	IR/CAS/protocoles	JSON Schema, JCS, unités/repères, CAS atomique, 20 graphes
S2	Imports verticaux	STEP, DICOM et LAS avec authority, hashes, pertes et phantoms
S3	Compute et IA	Même kernel CPU/CUDA/HIP ; DLPack sûr ; instrumentation copies/sync
S4	SALOME et MED	Agent 9.16, ProbeCapabilities, SMESH/MED, groupes, champs, >2 Gio
S5	HPC et MVX	Bundle Slurm/Apptainer/checkpoint ; 30-50 objets MVX
S6	Intégration et décision	Rapports, SBOM, benchmarks, défauts, démonstration rejouable, Gate G1

16.1 Backlog initial - dix jours SALOME

- Lancer SALOME 9.16 sans GUI ; tester session-less puis session terminal standard.
- Implémenter ProbeCapabilities avec versions, modules, MED, meilleurs OSS, limites et licences.
- Comparer le même STEP par OCCT direct et SHAPER/GEOM sur unités, placements, topologie, aire et volume.

- Créer un mesh SMESH libre, groupes et hypothèses ; exporter en MED sans MeshGems.
- Relire MED par MEDCoupling/MEDLoader : vecteurs, nœuds, cellules, Gauss et séries temporelles.
- Traiter un artefact supérieur à 2 Gio uniquement par URI/hash.
- Tester STEP/XAO/Study HDF et la faisabilité du dump SHAPER headless.
- Publier ADR, SBOM, rapport différentiel et décision Go/No-Go.

16.2 Definition of Done P0

Terminé signifie prouvé

Code fusionné sur branche protégée, tests associés, CI verte, documentation, artefacts hashés, licences auditées, métriques brutes conservées et exigence marquée avec sa preuve. Une simple capture d'écran ou un notebook non rejouable ne satisfait pas la Definition of Done.

17. Gates Go/No-Go

17.1 Gate G1 - fin P0

- 20 graphes IR validés, canonisés et rejoués.
- Imports STEP, DICOM et LAS/LAZ avec unités, repères et diagnostics corrects.
- Même kernel sur CPU, CUDA et HIP dans les tolérances publiées.
- DLPack sûr avec copies, synchronisations et durée de vie instrumentées.
- Bundle Slurm/Apptainer avec checkpoint et provenance.
- Agent SALOME headless ; zéro dépendance SALOME/Qt/PyQt/omniORB dans core.
- Groupes SMESH conservés dans le profil testé et aucun MeshGems.
- Artefact >2 Gio transporté uniquement par URI/hash.
- Micro-corpus MVX de 30 à 50 objets complètement analysé.
- Zéro non-conformité bloquante ou critique.

No-Go ou recentrage

Dépendance propriétaire obligatoire, perte silencieuse d'unité ou de repère, dépendance SALOME dans le cœur, groupes incomplets, impossibilité d'exécuter HIP, ou payload massif dans le contrôle. Un échec MVX arrête uniquement la trajectoire MVX/JEPA.

17.2 Gates G2 à G5

Gate	Critères essentiels
G2 / P1	CLI/Python, imports STEP/DICOM/LAS, MED/XAO vers IR, workers CPU/CUDA/HIP, DLPack/Arrow/ONNX, bundle Slurm, ~1 000 MVX, build reproductible.
G3 / P2	Connecteurs ParaView/Slicer/FreeCAD, SDK C v1-rc, SALOME bidirectionnel, PETSc/MFEM, deux cas utilisateurs engagés.
G4 / P3	NVIDIA + AMD, idéalement LUMI/Leonardo, reprise, I/O parallèle, audit sécurité/licences, trois cas externes reproductibles.
G5 / P4	ABI C v1 gelée, profils et corpus publics, Linux x86-64/ARM64 reproductibles, zéro Sev 1/2, gouvernance et relève effectives.

18. Livrables contractuels

Chaque tranche livre au minimum les éléments suivants, contrôlés dans un dépôt détenu par le maître d'ouvrage.

#	Livrable	Condition d'acceptation
1	Sources complets	Tag public, historique, DCO et aucune dépendance à un compte privé.
2	Packages/binaires	Reconstructibles, checksums, signature et matrice de plateformes.
3	Spécifications	Schémas, profils, ADR, migrations et exemples valides/invalides.
4	Tests et corpus	Manifestes de licence, générateurs, expected invariants et résultats bruts.
5	Rapport de conformité	Statut exigence par exigence, preuve et waiver éventuel.
6	Rapport de performance	Méthode, matériel, répétitions, données brutes et interprétation.
7	Sécurité	Modèle de menace, scans, fuzzing, défauts et dérogations.
8	SBOM/licences	SPDX ou CycloneDX, notices et obligations par artefact.
9	Documentation	Installation, API, CLI, formats, exploitation, migration et limites.
10	Démonstration rejouable	Script ou bundle complet, sans étape manuelle cachée.
11	Dossier de transfert	Architecture, runbooks, formation et accès transférés.

#	Livrable	Condition d'acceptation
12	Liste des défauts	Sévérité, reproduction, propriétaire, échéance et décision.

Aucun livrable NE DOIT dépendre d'un dépôt, runner, compte cloud, secret ou licence détenu exclusivement par un prestataire.

19. Équipe, RACI et charge

Rôle	Responsabilité	Charge indicative
Product Owner / autorité domaine	Périmètre, priorités, gates, arbitrage médical et acceptation	Dr Olivier Ami
Architecte principal	IR, ABI, frontières, ADR et qualité globale	1 ETP
Lead CAO/CAE	OCCT, STEP, SALOME, MED, fidélité géométrique	0,8-1 ETP
Lead données médicales	DICOM, phantoms, confidentialité et Slicer	0,5-0,8 ETP
Lead GPU/HPC	Kokkos, workers, Slurm, performance et EuroHPC	0,8-1 ETP
Python/build/packaging	CLI, API, CI, packages et documentation	0,8 ETP
QA/conformité	Tests, corpus, traçabilité et recette indépendante	0,6-1 ETP
Sécurité/licences	Threat model, SBOM, audit et release	0,2-0,4 ETP
Partenaires	Hardware NVIDIA/AMD/ARM64, centre HPC, design partners	Ponctuel

19.1 Répartition des 60 personnes-mois

Domaine	Personnes-mois
Core et architecture	12
Géométrie, CAE et SALOME	10
Données scientifiques et DICOM	10
GPU, HPC, IA et MVX	12
Python, CI et QA	10
Sécurité, licences, documentation et corpus	6

20. Budget indicatif et leviers de réduction

Les montants ci-dessous traduisent l'effort en coût externe pour permettre un arbitrage. Ils ne créent aucune obligation commerciale et n'incluent pas une qualification médicale. Hypothèse : 20 jours par personne-mois et 700 à 1 050 euros HT par jour.

Périmètre	Jours	Main-d'œuvre externe	Enveloppe avec 15-25 %
P0	≈ 120	84-126 k€	97-158 k€
P0 + P1	≈ 320	224-336 k€	258-420 k€
Jusqu'à P2	≈ 560	392-588 k€	451-735 k€
Jusqu'à P3	≈ 800	560-840 k€	644 k€-1,05 M€
Jusqu'à v1	≈ 1 200	840 k€-1,26 M€	≈ 1,0-1,55 M€

Un consortium académique et open source peut abaisser la dépense monétaire grâce aux contributions en nature et aux accès HPC, sans supprimer l'effort réel. Une enveloppe monétaire de 350 à 750 k€ reste plausible si une part importante des 60 personnes-mois est apportée en nature.

20.1 Économies déjà intégrées

Décision	Économie estimée	Compromis
Pas de GUI propre au MVP	6-12 pers.-mois	Branding et workflow unifié différés.
VTk/ParaView d'abord	6-18 pers.-mois	Contrôle viewport fin différé.
OCCT plutôt qu'un kernel maison	Plusieurs années	Dépendance à la maturité amont.
Kokkos/PETSc	12-24 pers.-mois	Spécialisations vendor possibles après profilage.
Workers ciblés	3-6 pers.-mois	Pas de binaire GPU universel.
Monorepo du socle	Coordination/CI réduites	Séparations tardives disciplinées.
Recettes vendor/SALOME	Audit et redistribution réduits	Installation locale demandée.

06

Vérification et recette

Corpus, métriques, traçabilité, anomalies et release gates

21. Stratégie de vérification

Code	Méthode	Preuve recevable
T	Test	Test automatisé, environnement, résultat brut et artefact hashé.
I	Inspection	Revue de code, binaire, licence, document ou configuration.
A	Analyse	Calcul, statistique, benchmark, threat model ou justification d'écart.
D	Démonstration	Scénario rejouable par un tiers ; jamais seule pour une propriété interne.

21.1 Chaîne de traçabilité

```
REQ-ID
-> besoin utilisateur / source
-> ADR et composant
-> lot de travaux, issue et pull request
-> TEST-ID et environnement
-> exécution CI et données brutes
-> artefact hashé
-> procès-verbal d'acceptation
```

21.2 Source de vérité des exigences

```
id: MOR-IR-001
title: JSON Schema authority
shall: Le manifeste normatif DOIT utiliser JSON Schema 2020-12.
rationale: Validation indépendante et outillage multi-langage.
priority: MUST
phase: P0
owner: architecture
verification_method: T
test_ids: [IR-SCHEMA-001, IR-SCHEMA-002]
deliverables: [ir.schema.json, validation-report.json]
dependencies: []
status: approved
waiver: null
```

22. Corpus de conformité

Corpus	Volume minimal	Contrôles
Graphes IR	20	Validation, canonicalisation, migrations, extensions inconnues
STEP/AP242	20 P0 ; 100 P1	Unités, placements, assemblages, noms, couleurs, topologie, aire, volume
SMESH	20	Types/ordres, connectivité, groupes, hypothèses, qualité
MED	10	Nœuds, cellules, Gauss, composantes, unités, nature, temps
DICOM	20 P0 ; 100 P1	UID, FoR, géométrie multi-frame, SEG/SR/RT, pixels et de-id
Points/meshes	50+	CRS, précision, attributs, LOD, qualité
Gros artefacts	3 x >2 Gio	Streaming, URI/hash, reprise, mémoire et timeout
MVX faisabilité	30-50	Pipeline, métriques, seuils et catégories d'échec
MVX validation	≈ 1 000	Distribution d'erreurs, invariances, baselines et coût
Robustesse	10 000+	Mutations, malformed inputs, fuzz-like et out-of-core

22.1 Manifeste d'un actif de conformité

```
asset_id, sha256, source, license, classification,
redistribution_status, format_profile,
expected_invariants, tolerances, allowed_losses,
expected_diagnostics, generator_version
```

22.2 Mesures obligatoires par domaine

Domaine	Mesures
CAO/B-Rep	Validité, comptes topologiques, distance, aire/volume, assemblages, attributs du profil.
Mesh	Chamfer/Hausdorff normalisés, normales, manifold, watertight, auto-intersections, qualité éléments.
Points	Compte, CRS, attributs, précision, couverture, densité et quantiles de distance.
DICOM	Relations UID, FoR, orientation, origine, spacing, intensités, nombre de frames.
Champs	L2/L∞, conservation, unités, support, interpolation, temps et précision.
IA	Pointeur, allocations, copies, synchronisations, dtype/layout et provenance du modèle.
HPC	Temps, scaling, efficacité, I/O, checkpoint, reprise, énergie si disponible.
MVX	Chamfer, Hausdorff, F-score, IoU, normales, topologie, validité et coût.

23. Classification et acceptation des anomalies

Sévérité	Définition	Acceptation de phase
Sev 1 - bloquant	Corruption, perte de données, faille critique, résultat silencieusement faux	Interdite
Sev 2 - majeur	Exigence MUST inutilisable sans contournement	Interdite
Sev 3 - modéré	Contournement documenté acceptable	Possible avec propriétaire, échéance et accord écrit
Sev 4 - mineur	Présentation, ergonomie ou documentation non bloquante	Possible et planifiée

L'acceptation d'une tranche exige zéro Sev 1 et zéro Sev 2. Une anomalie qui modifie silencieusement une unité, un repère, une topologie, un support de champ ou une autorité est Sev 1.

23.1 Waivers

- Un waiver désigne l'exigence, le périmètre exact, la justification, le risque, le propriétaire et l'expiration.
- Un waiver ne peut pas masquer une perte silencieuse, une corruption, une faille critique ou une obligation de licence.
- Un waiver expiré redevient une non-conformité bloquante.
- Le Product Owner approuve les waivers fonctionnels ; sécurité et licence exigent aussi l'avis du référent concerné.

24. Matrice CI et release

Tier	Plateformes	Cadence	Bloque
Tier 1	Linux x86-64 CPU ; CUDA ; HIP ; Linux ARM64 CPU/CUDA	PR ou nightly selon coût GPU	Merge/release
Tier 2	Windows x86-64 CPU/Vulkan ; Linux SYCL	Hebdomadaire et RC	Release à partir de P3
Tier 3	macOS ARM64 ; OpenCL ; WebGPU	Best effort	Non
HPC	LUMI, Leonardo, JUPITER ou équivalents	Jalons P2/P3 et RC	Gate concerné
SALOME	9.16 Linux x86-64	Nightly et RC P0-P2	Bridge et io-med

24.1 Release checklist

- Tous les MUST de la phase ont une preuve et aucun waiver interdit.
- CI Tier 1 verte, tests longs exécutés, couverture atteinte et fuzzing sans crash reproductible.
- SBOM, licences, vulnérabilités, signatures et hashes publiés.
- Benchmarks comparables à la baseline et régressions supérieures à 10 % approuvées.
- Documentation versionnée, exemples exécutés et limitations visibles.
- Packages reconstruits depuis le tag dans un environnement propre.
- Dossier des défauts, waivers et migrations publié.

07

Gouvernance et risques

Décisions, dépendances, signaux d'arrêt et démarrage immédiat

25. Registre de risques

Risque	Réduction	Signal d'arrêt ou recentrage
Périmètre excessif	Tranches, anti-scope, ADR et propriétaire	Domaine ajouté sans utilisateur ni mainteneur
IR nouveau silo	Exports standards et schémas indépendants	Données inutilisables hors Morphoia
Fidélité insuffisante	Profil, corpus, métriques et pertes	Réparation silencieuse ou unité ambiguë
Identités topologiques	Confiance et stratégie explicites	Promesse de stabilité non démontrable
Bridge SALOME fragile	Agent isolé et artefacts standards	Dépendance du cœur à CORBA/Qt
Portabilité GPU	Trois kernels P0 et CPU référence	HIP impossible ou écart >2x durable
Données de santé	Phantoms, de-id, logs contrôlés	PHI dans CI, logs ou artefacts publics
Licences	SBOM et audit par composant	Dépendance non redistribuable obligatoire
Accès HPC	Partenaire et profils avant P2	Aucun environnement AMD/NVIDIA disponible
Corpus	Générateurs et licences par actif	Aucune conformance minimale publiable
Bus factor	CODEOWNERS, suppléants et transfert	Module critique détenu par une seule personne
MVX non concluant	Micro-corpus avant 1 000	Reconstruction non utile pour l'objectif IA
Supply chain	Locks, signature et provenance	Artefacts impossibles à reproduire
Non-adoption	Design partners et cas jouables	Aucun utilisateur externe actif après P1

26. ADR initiaux

ADR	Décision	Statut
ADR-001	Plateforme fédérée, pas moteur monolithique	Accepté
ADR-002	C++20 interne, ABI C publique, Python utilisateur	Accepté
ADR-003	IR JSON canonique + CAS ; aucun format binaire universel	À valider P0
ADR-004	OCCT/XDE et profil STEP/AP242	Accepté
ADR-005	VTK/ITK/PDAL pour les représentations scientifiques	Accepté
ADR-006	Workers Kokkos CPU/CUDA/HIP ; SYCL Tier 2	À valider P0
ADR-007	DLPack + Arrow C Device pour l'IA	À valider P0
ADR-008	VTK d'abord ; Vulkan/ANARI/Hydra optionnels	Accepté
ADR-009	HPC via Slurm/Spark/Apptainer/ADIOS2	À valider P0
ADR-010	Apache-2.0 OR MIT ; aucune clause NC	Accepté sous audit
ADR-011	SALOME backend optionnel et oracle, jamais dépendance du cœur	Accepté
ADR-012	MEDCoupling dans un paquet LGPL dynamique séparé	Accepté sous audit
ADR-013	STEP/XAO/MED transportent les artefacts ; CORBA reste interne	Accepté
ADR-014	DAG Morphoia/Slurm autoritaires ; YACS/JOBMANAGER hôtes	Accepté
ADR-015	Monorepo permissif pour le socle P0-P1	Proposé par ce cahier des charges
ADR-016	MVX possède un gate indépendant et ne conditionne pas le socle	Proposé par ce cahier des charges

27. Dépendances externes à sécuriser

- Une machine NVIDIA, une machine AMD ROCm et une CPU ARM64 avant la fin de P1.
- Un partenaire universitaire ou EuroHPC pour profils Slurm, Spack et CI jalonnée.
- Référents OCCT/STEP, DICOM, VTK/ParaView, Kokkos/PETSc et sécurité C++.
- Corpus redistribuables ou générateurs synthétiques pour chaque profil prioritaire.
- Deux design partners avant P2 et trois cas utilisateurs externes avant G4.
- Un référent licences pour SALOME, MEDCoupling, codecs et distributions concrètes.

27.1 Conditions préalables au jour 1

Pré-requis	Responsable	Échéance
Dépôt principal et contrôle des accès	Product Owner / tech lead	Jour 1
Choix des responsables de domaine	Product Owner	Jour 3
Runners CPU et politique des secrets	Build/CI	Jour 3
Machine ou runner NVIDIA + AMD réservé	GPU/HPC	Semaine 2
SALOME 9.16 installé et hashé	CAE	Jour 3
Actifs P0 juridiquement validés	QA/licences	Semaine 1
Machine de référence performance	QA/performance	Semaine 1

28. Plan d'action des 30 premiers jours

Ordre	Action	Résultat
1	Baseliner ce cahier des charges et exporter requirements.yaml	Catalogue versionné et assignable
2	Créer monorepo, licences, DCO, gouvernance et CI CPU	Socle public minimal
3	Publier ADR-001 à 016 et anti-scope	Arbitrages non ambigus
4	Implémenter IR/CAS minimal et 20 graphes	Contrat exécutable
5	Importer un STEP, une série DICOM et un LAS/LAZ	Premier vertical data
6	Porter trois kernels P0 sur CPU/CUDA/HIP	Décision Kokkos mesurée
7	Démontrer DLPack et tracer copies/synchronisations	Data plane IA validé
8	Conduire le spike SALOME 9.16	Bridge et oracle Go/No-Go
9	Exécuter bundle Slurm/Apptainer avec checkpoint	Chemin HPC initial
10	Traiter 30-50 objets MVX et geler les seuils	Décision avant corpus 1 000
11	Lancer les entretiens utilisateurs sur le démonstrateur	Backlog P1 validé
12	Préparer rapport G1, SBOM et dérogations	Décision fin P0

Décision de lancement

Le projet peut être lancé sur TF0/P0 avec ce document comme baseline. Les choix non encore verrouillés - versions exactes, seuils MVX définitifs, machines de référence et tolérances par fixture - sont précisément des livrables des deux premières semaines, pas des ambiguïtés laissées au développement.

A

Annexes techniques

Pile, formats, conformité, glossaire et références

Annexe A. Pile logicielle de référence

Domaine	Brique	Licence indicative	Intégration
Langage/API	C++20 ; Python 3.13 ref. / 3.12 compat.	ISO / PSF	Core + API utilisateur
Géométrie	OCCT 8.0.1 / OCAF / XDE	LGPL-2.1 + exception	Plugin natif dynamique
CAE data	MEDCoupling/MEDLoader	LGPL-2.1-or-later	Paquet séparé
MED bas niveau	MED-fichier 6.0.1	LGPL-3.0-or-later	Dépendance io-med
SALOME	9.16 ; Python 3.9 ; OCCT 7.9 patché	Composition multilicence	Agent externe
Visualisation	VTK 9.6.2 / ParaView	BSD-3-Clause	Natif + plugin
Imagerie	ITK 5.4.6 LTS	Apache-2.0	Natif
DICOM	DCMTK + GDCM	BSD-like / BSD	Module dédié
Points	PDAL 2.10.2	BSD-3-Clause	Module dédié
Volumes creux	OpenVDB/NanoVDB	Apache-2.0	Option P2
Scènes	OpenUSD 26.08 / MaterialX	TOST-1.0 / Apache-2.0	Adapter optionnel
Kernels	Kokkos 5.2.0 / Kokkos Kernels	Apache-2.0 + exception LLVM	Workers
Solveurs	PETSc ; MFEM/libCEED	BSD	Plugins P2
IA	PyTorch ; ONNX Runtime 1.28.0	BSD-3 / MIT	Runtimes externes
Échange	DLPack 1.3 ; Arrow 25.0.0	Apache-2.0	ABI mémoire
I/O HPC	ADIOS2 2.12.1	Apache-2.0	Natif
Cluster	Slurm 26.05 ; MPI 5.0 / UCX site	GPL-2.0+ / variables	Client/site
Packaging	Spack 1.2.2 ; Apptainer 1.5.3	Apache-2.0 OR MIT / BSD-3	Outils

Les licences sont indicatives et ne remplacent pas l'audit du tag, des options, codecs, patches et artefacts réellement distribués. CUDA/cuDNN et SDK propriétaires restent des recettes utilisateur.

Annexe B. Matrice de formats et objectifs

Format	Lecture	Écriture	Moteur	Niveau cible
STEP AP242 profil	Oui	Oui	OCCT/XDE	A/B après corpus
STEP AP214/AP203	Oui	Oui	OCCT	B
IGES 5.3	Oui	Oui	OCCT	B
OCCT BREP	Oui	Oui	OCCT	A versionnée
STL/OBJ/PLY	Oui	Oui	VTK/OCCT	A/B selon attributs
gITF/GLB	Oui	Oui	OCCT/OpenUSD	A dérivé
VTK XML/VTKHDF	Oui	Oui	VTK	A par profil
MED	Oui	Oui	MEDCoupling/agent	A/B
XAO	Oui	Oui	SALOME agent	B spécialisé
Study HDF	Opaque	Agent	SALOME	C/D
LAS/LAZ/COPC	Oui	Oui/partiel	PDAL	A/B
DICOM image	Oui	Non clinique	DCMTK/ITK	A/B
DICOM SEG/SR/RT	Profil	Profil	DCMTK/Slicer	B
NIFTI	Oui	Oui	ITK	A dérivé
OpenVDB	Oui	Oui	OpenVDB	A
OpenUSD	Oui	Oui	OpenUSD	B dérivé
MVX LP3	Oui	Oui	Morphoia	D expérimental
CATIA/NX/Creo/SW	Plugin/SDK	Plugin/SDK	Outil source	D

Annexe C. Résumé du catalogue d'exigences

Famille	Total	MUST	SHOULD	MAY	Phases
SCP	15	14	1	0	P0, P2+
ARC	14	13	0	1	P0-P1, P2+
DEV	16	16	0	0	P0-P1
API	18	18	0	0	P0-P1, P4
IR	18	18	0	0	P0-P1
CAS	12	11	0	1	P0-P1
IO	19	19	0	0	P0-P2, P2
DIC	14	14	0	0	P0-P1, P2
VIS	10	7	0	3	P1, P2, P2-P3
CMP	19	18	1	0	P0-P2, P2, P3, P3+
AI	17	17	0	0	P0-P1
MVX	18	16	2	0	P0-P1
PLG	10	10	0	0	P0-P3, P1-P2
SAL	27	27	0	0	P0-P2, P1-P2, P2
HPC	16	15	1	0	P0-P3, P3
SEC	20	18	2	0	P0-P2, P2
PER	16	13	3	0	P0-P3
QA	18	18	0	0	P0-P4, P3
LIC	13	13	0	0	P0-P1
GOV	10	10	0	0	P0-P2, P2+

Total normatif : **320 exigences**. Le tableau détaillé des pages précédentes constitue la baseline humaine ; requirements.yaml doit devenir la baseline machine en P0.

Annexe D. Glossaire

Terme	Définition
ABI	Contrat binaire entre composants compilés ; l'ABI C limite les ruptures C++.
B-Rep	Représentation exacte par frontières : faces, arêtes, sommets et surfaces paramétriques.
CAS	Stockage adressé par contenu ; un bloc immuable est identifié par son hash.
DAG	Graphe orienté acyclique de dépendances, opérations et provenance.
DLPack	Contrat mémoire minimal pour partager des tenseurs entre runtimes.
FoR	DICOM Frame of Reference, référentiel spatial partagé par des objets médicaux.
IR	Intermediate Representation fédérant identité, sémantique et provenance.
LOD	Level of detail, représentations de précision variable pour interaction/streaming.
MEDCoupling	Bibliothèque C++/Python de meshes, champs, interpolation et transferts.
MVX	Famille expérimentale d'encodages multi-vues Morphoia évaluée par round-trip.
OCAF/XDE	Frameworks OCCT de documents, assemblages et métadonnées d'échange.
Oracle différentiel	Implémentation de comparaison ; une divergence déclenche une analyse.
Round-trip	Import puis export/reconstruction avec conservation mesurée du profil.
SBOM	Nomenclature des composants logiciels, versions, licences et dépendances.
Worker	Binaire headless ciblé sur un backend et consommant un job bundle.
XAO	Artefact SALOME spécialisé liant forme, topologie, groupes et champs.
Zero-copy	Partage du même buffer sans duplication ; dépend du device, layout et sync.

Annexe E. Références officielles

ID	Source	Lien / note
R01	Morphoia Engine - Architecture de référence v0.2	Document local, 6 août 2026
R02	SALOME Platform 9.16 - annonce et notes de version	https://www.salome-platform.org/
R03	SALOME KERNEL - architecture et modes d'exécution	https://docs.salome-platform.org/latest/tui/KERNEL/running_salome_page.html
R04	MEDCoupling - documentation développeur	https://docs.salome-platform.org/latest/dev/MEDCoupling/developer/index.html
R05	SMESH - dépôt officiel	https://github.com/SalomePlatform/smesh
R06	SHAPER - documentation	https://docs.salome-platform.org/latest/gui/SHAPER/General/Introduction.html
R07	OCCT - interface STEP	https://dev.opencascade.org/doc/overview/html/occt_user_guides__step.html
R08	OCCT - XDE	https://dev.opencascade.org/doc/overview/html/occt_user_guides__xde.html
R09	VTK - documentation et licence	https://docs.vtk.org/en/latest/about.html
R10	ITK - documentation	https://docs.itk.org/en/latest/
R11	PDAL - documentation	https://pdal.io/
R12	DICOM Standard - édition courante	https://www.dicomstandard.org/current/
R13	Kokkos - configuration des backends	https://kokkos.org/kokkos-core-wiki/get-started/configuration-guide.html
R14	PETSc - installation et backends	https://petsc.org/release/install/install/
R15	MFEM - fonctionnalités	https://mfem.org/features/
R16	Khronos Vulkan	https://www.khronos.org/vulkan/
R17	Khronos ANARI	https://www.khronos.org/anari/
R18	OpenUSD - documentation	https://openusd.org/
R19	DLPack - spécification	https://dmlc.github.io/dlpack/latest/python_spec.html
R20	Arrow C Device Data Interface	https://arrow.apache.org/docs/format/CDeviceDataInterface.html
R21	ONNX Runtime - Execution Providers	https://onnxruntime.ai/docs/execution-providers/
R22	Slurm - overview	https://slurm.schedmd.com/overview.html
R23	Spack - environnements	https://spack.readthedocs.io/en/latest/environments.html
R24	Apptainer - support GPU	https://apptainer.org/docs/user/main/gpu.html
R25	ADIOS2 - documentation	https://adios2.readthedocs.io/en/latest/
R26	EuroHPC - supercalculateurs	https://www.eurohpc-ju.europa.eu/supercomputers/our-supercomputers_en
R27	JSON Schema Draft 2020-12	https://json-schema.org/draft/2020-12
R28	RFC 8785 - JSON Canonicalization Scheme	https://www.rfc-editor.org/rfc/rfc8785
R29	RFC 9562 - UUID	https://www.rfc-editor.org/rfc/rfc9562
R30	UCUM - Unified Code for Units of Measure	https://ucum.org/
R31	Semantic Versioning	https://semver.org/
R32	REUSE - conformité des licences	https://reuse.software/
R33	SPDX - spécifications	https://spdx.dev/
R34	CycloneDX - SBOM standard	https://cyclonedx.org/
R35	SLSA - supply-chain levels	https://slsa.dev/
R36	NIST Secure Software Development Framework	https://csrc.nist.gov/Projects/ssdf
R37	Open Source Definition	https://opensource.org/osd
R38	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
R39	Developer Certificate of Origin	https://developercertificate.org/
R40	CMake Presets	https://cmake.org/cmake/help/latest/manual/cmake-presets.7.html
R41	Python - statut des versions	https://devguide.python.org/versions/
R42	OCCT 8.0.1 - annonce officielle	https://dev.opencascade.org/content/open-cascade-technology-801-release
R43	ISO 10303-242:2025	https://www.iso.org/standard/84300.html
R44	VTK 9.6.2 - téléchargement	https://vtk.org/download/

ID	Source	Lien / note
R45	ITK - releases officielles	https://github.com/InsightSoftwareConsortium/ITK/releases
R46	PDAL - releases officielles	https://github.com/PDAL/PDAL/releases
R47	SALOME 9.16 - notes de version	https://www.salome-platform.org/wp-content/uploads/2026/07/SALOME_9_16_0_Release_Notes.pdf
R48	Kokkos - releases officielles	https://github.com/kokkos/kokkos/releases
R49	OpenUSD - releases officielles	https://github.com/PixarAnimationStudios/OpenUSD/releases
R50	OpenUSD - licence TOST 1.0 et notices	https://github.com/PixarAnimationStudios/OpenUSD/blob/dev/LICENSE.txt
R51	Apache Arrow - releases officielles	https://github.com/apache/arrow/releases
R52	ONNX Runtime - releases officielles	https://github.com/microsoft/onnxruntime/releases
R53	ONNX Runtime - provider MIGraphX	https://onnxruntime.ai/docs/execution-providers/MIGraphX-ExecutionProvider.html
R54	Slurm 26.05 - release notes	https://slurm.schedmd.com/release_notes.html
R55	MPI Forum - documents MPI 5.0	https://www.mpi-forum.org/docs/
R56	Apptainer - releases officielles	https://github.com/apptainer/apptainer/releases
R57	Spack - releases officielles	https://github.com/spack/spack/releases
R58	SPDX 3.0.1	https://spdx.dev/use/specifications/
R59	CycloneDX 1.7 - spécification	https://cyclonedx.org/specification/overview/
R60	SLSA 1.2	https://slsa.dev/spec/v1.2/
R61	in-toto Statement v1	https://in-toto.io/Statement/v1
R62	CISA - Minimum Elements for SBOM 2026	https://www.cisa.gov/resources-tools/resources/2026-minimum-elements-software-bill-materials-sbom

Conclusion

Morphoia Engine peut être lancé immédiatement comme programme à tranches. Le présent cahier des charges protège le projet contre les deux risques dominants : réécrire des briques déjà matures et confondre une démonstration avec une preuve de fidélité. La v1 est acceptée si un tiers peut vérifier l'identité, comprendre les pertes, rejouer les traitements et changer d'infrastructure sans changer la sémantique.